Data Analytics Capstone

Governing Agentic AI Coding Assistants in the Software Industry: Building and Evaluating an Ethics and Governance Framework Using Public Vulnerability Data

Final Report

Dr. Ramesh Babu Chellappan (Group 3)

Walsh College

QM640: Data Analytics Capstone

Instructor: Dr. Javad Katibai

Mentor: Mr. Jainesh Garg

Term 3

November 10, 2026

GitHub Repository

The data files, code, coded data set, database, model artefacts, and every table and figure in this report are available in the public repository https://github.com/RameshBabuChellappan/aceg-coding-agent-governance. The final-report materials sit in the following folders: the frozen source data (data/processed/cases_to_code.csv), the two completed human coding workbooks (coding/completed/), the large language model codes (data/llm/ and data/processed/llm_codes.csv), the final labelled data set (data/processed/cases_coded.csv), the SQLite database (data/aceg.db), the analysis and deployment scripts (src/07_parse_llm_codes.py to src/10_scorecard_service.py), the trained models (models/), and all outputs (outputs/final/). Appendix E lists the steps that reproduce every result.

Abstract

Artificial intelligence (AI) coding agents such as Claude Code, Cursor, and GitHub Copilot run commands and edit code with little oversight, yet software organizations cannot tell which ethics and governance controls fail or which failures matter most. This study built the Agentic Coding Ethics and Governance (ACEG) Framework, with eight domains and 20 controls mapped to Open Worldwide Application Security Project (OWASP) and National Institute of Standards and Technology (NIST) guidance, and evaluated it with statistical tests, machine learning, a neural network, and an agentic large language model (LLM) coder. The data were a census of $N = 223$ public coding-agent security advisories (unit: one advisory or Common Vulnerabilities and Exposures [CVE] record) published from October 31, 2023, to October 10, 2026, in the GitHub Advisory Database and CVE List V5. Two human coders and a blinded LLM coder labelled every case. Analysis used Python 3.13 (scikit-learn, SciPy, SQLite, Flask) on a 2-core Intel Xeon central processing unit (CPU) without a graphics processing unit (GPU). ACEG covered 99.6% of cases (95% confidence interval [CI] [97.5, 99.9]), and agency (66.8%) and input-trust (51.6%) failures dominated. Domain family was unrelated to whether exploitation needed user interaction, $\chi^2(3) = 1.42$, $p = .700$. The Governance Compliance Score did not significantly predict a Common Vulnerability Scoring System (CVSS) High or Critical rating (odds ratio [OR] = 1.34 per standard deviation, $p = .057$), but the number of breached domains did (OR = 2.28, $p < .001$). A random forest on the ACEG control profile plus weakness type performed best (cross-validated area under the curve [AUC] = .85; hold-out AUC = .84), ahead of the score-based logistic model (.69) and weakness-type triage (.70). The LLM coder agreed with the human consensus at $\kappa = .58$, 95% CI [.53, .63], below the .60 target but double human–human agreement ($\kappa = .28$). The solution is packaged as the ACEG Governance Skill, an industry-ready agent skill and

application programming interface (API) that codes, scores, and plans remediation for new agent advisories; fully automated, it reached a hold-out AUC of .81.

Keywords: agentic AI, coding agents, AI governance, vulnerability data, LLM annotation, random forest, inter-rater reliability

Governing Agentic AI Coding Assistants in the Software Industry

Introduction

Software teams are giving artificial intelligence (AI) agents real authority. Coding assistants that once offered single-line completions now plan multistep changes, call external tools, and act on a developer's machine or a build server. When such an agent misreads its instructions, follows a hidden command in a repository file, or runs with broader permissions than intended, the result is not just a bug. It is a lapse in accountability, consent, and control. This report shows how often those lapses occur in disclosed weaknesses, which governance controls they involve, whether control failures go with more serious weaknesses, and whether a large language model (LLM) can apply the governance rulebook as reliably as people can.

Background and Context

The setting is the software industry, where coding agents such as Claude Code, Cursor, GitHub Copilot, OpenAI Codex, Gemini command-line interface (CLI), Cline, and Kiro read whole repositories, run terminal commands, and connect to tools through the Model Context Protocol (MCP; Hou et al., 2025). Benchmarks such as SWE-bench show that agents built on transformer language models can now resolve real GitHub issues end to end (Jimenez et al., 2024; Vaswani et al., 2017), and interface designs such as SWE-agent widen what they can do (Yang et al., 2024). The stakeholders are engineering leaders and platform teams who choose and configure agents, security and AI-governance functions who approve them, risk committees who answer for incidents, and vendors who build the tools. In 2025, a Replit agent deleted a production database during a declared code freeze (Responsible AI Collaborative, 2025). The case showed that failures of agent autonomy are operational and reputational events, not hypothetical ones.

Governance guidance exists but is hard to apply to coding agents. The AI Risk Management Framework (AI RMF) of the National Institute of Standards and Technology

(NIST) defines govern, map, measure, and manage functions (NIST, 2023). The International Organization for Standardization and the International Electrotechnical Commission (ISO/IEC) standard 42001 sets requirements for an AI management system (ISO/IEC, 2023). The Open Worldwide Application Security Project (OWASP) Top 10 for Agentic Applications names the leading agent risks (OWASP Gen AI Security Project, 2025). None of these instruments gives a compact, testable control set for coding agents, and none has been checked against field evidence. Research on agent governance argues that autonomy creates accountability gaps and recommends practices such as approval gates and attributable actions (Chan et al., 2023; Shavit et al., 2023), but it does not measure how often those practices fail in shipped products.

Field evidence does exist. When a coding agent has a security weakness, the vendor or a vulnerability numbering authority publishes an advisory with a Common Vulnerabilities and Exposures (CVE) identifier, a Common Weakness Enumeration (CWE) type, and a severity score from the Common Vulnerability Scoring System (CVSS; Forum of Incident Response and Security Teams [FIRST], 2023). Disclosures have grown quickly: 155 of the 223 eligible advisories studied here were published in 2026 alone. This makes it possible to treat agent governance as a data analytics problem, scoring each disclosed weakness against an explicit control framework and analyzing the body of evidence as a whole instead of one advisory at a time.

Problem Statement

Software organizations adopt AI coding agents without a measurable, evidence-based way to judge which ethics and governance controls those agents breach, how serious the resulting weaknesses are, or whether compliance can be assessed reliably at scale. The objective of this study is to *predict and explain* Y = whether a disclosed coding-agent weakness is high-impact (CVSS High or Critical, 1/0) for each public security advisory (the

unit of analysis) using X = the governance profile of the case under the Agentic Coding Ethics and Governance (ACEG) Framework (the Governance Compliance Score [GCS], the eight governance domains, and the 20 control-breach indicators, with component type, number of domains, publication year, and CWE group as covariates) over advisories published from January 1, 2023, to the data freeze of October 11, 2026. Success is evaluated with the hold-out and cross-validated area under the receiver operating characteristic (ROC) curve (AUC), balanced accuracy, F1, and Brier score for prediction; with Wilson 95% confidence intervals (CIs), chi-square tests, and odds ratios (ORs) at $\alpha = .05$ for the hypotheses; and with Cohen's κ against human consensus (target $\kappa > .60$) for automated coding.

Purpose of the Study

The study follows design-science logic: it builds an artifact, applies it to real data, evaluates it, and refines it (Hevner et al., 2004). Its purpose has four parts. It *estimates* how much of the disclosed evidence the ACEG Framework covers and how common each governance domain is. It *tests an association* between the family of the main governance failure and the need for human interaction during exploitation. It *classifies and explains* high-impact cases with logistic regression, tree ensembles, and a neural network, and identifies which breached controls carry the predictive signal. Finally, it *automates and compares*: an LLM coder applies the same codebook as the human coders, and its labels are measured against theirs. The deliverables are a coded public data set, answers to four research questions, an industry-ready ACEG Governance Skill with a scorecard service, and an evidence-based ACEG Version 2.

Research Problems / Research Questions

Research Question 1 (RQ1): What proportion of disclosed coding-agent weaknesses does the ACEG Framework cover, and how prevalent is each of its eight governance domains?

Research Question 2 (RQ2): Is the domain family of a case's primary governance failure associated with whether exploiting the weakness requires human interaction?

Research Question 3 (RQ3): Does a lower GCS predict a high-impact case after adjusting for component type, number of domains, and publication year, and which model and features predict high impact best?

Research Question 4 (RQ4): How closely does an automated LLM coder reproduce human coding when both apply the same ACEG codebook?

Contributions and Expected Value

Practical contribution. The study gives software organizations a ready-to-use control framework: eight domains, 20 controls, and an OWASP and NIST crosswalk. The ACEG Governance Skill, a portable agent skill that any LLM agent platform can load, turns a newly disclosed agent weakness into governance codes, a governance score, a predicted probability of high impact, a priority tier, and a remediation plan with owners and verification checks.

Technical contribution. The study contributes a coded census of 223 coding-agent advisories, a reproducible SQLite pipeline, a head-to-head comparison of eight predictive models (logistic regression, random forest, gradient boosting, and a multilayer perceptron [MLP] neural network) under a hold-out split and repeated cross-validation, and a three-coder reliability design in which an LLM coder is scored only on cases where the two humans already agree.

Value to stakeholders. Platform and DevSecOps teams get triage priorities grounded in field evidence instead of vendor claims. AI-governance offices and auditors working to

ISO/IEC 42001 get controls they can test against artefacts. Agent vendors get a domain-level view of where their peers' products fail. For the research community, the human-versus-LLM results show where AI-assisted governance coding can be trusted and where human review must remain.

Literature Review

This section reviews prior work on the security of AI coding assistants, attacks on LLM agents, frameworks for governing agentic systems, the validity of vulnerability severity data, LLMs as annotators, and the choice of models for small tabular data. Each source is judged by how it shaped a research question or a design decision.

Literature Review Approach

Sources were found through Google Scholar, the Association for Computing Machinery (ACM) Digital Library, the Institute of Electrical and Electronics Engineers (IEEE) Xplore library, arXiv, and the publications of standards bodies. The search terms were *coding agent*, *AI code assistant security*, *prompt injection agent*, *Model Context Protocol security*, *agentic AI governance*, *LLM annotation reliability*, *LLM-as-a-judge*, *CVSS validity*, and *tree-based models tabular deep learning*. A source was included if it was peer-reviewed, a standard, or a technical report from a major laboratory; if it was published from 2019 onward (methodological classics excepted); and if it informed at least one research question or design choice. Titles and abstracts were screened against these criteria, and the 18 sources summarized in Table 1 were retained as key literature. Methodological references (for example, for sample size and κ) are cited where they are used.

Summary of Key Literature

Table 1

Literature Relevance Matrix

Author (Year)	Domain / Context	Dataset / Setting	Method(s)	Key Findings	RQ Linkage	How It Supports Project Decisions
Pearce et al. (2022)	Security of AI-generated code	89 scenarios; 1,689 Copilot programs	Scenario testing; static analysis (CodeQL)	About 40% of generated programs were vulnerable	RQ1	Motivated D6 and control C13 (security testing gate)
Perry et al. (2023)	Developers using AI assistants	User study, 47 participants	Randomized controlled experiment	Assisted users wrote less secure code yet felt more confident	RQ2	Human checkpoint (D8) tested as a variable, not assumed to protect
Greshake et al. (2023)	LLM-integrated applications	Real applications; proof-of-concept attacks	Threat taxonomy; demonstrations	Indirect prompt injection redirects goals without user action	RQ1, RQ2	Defined D4 and C08; user interaction (from CVSS) as RQ2 variable
Zhan et al. (2024)	Tool-using LLM agents	InjecAgent: 1,054 test cases, 30 agents	Benchmark evaluation	Most agents followed injected tool content	RQ1	Expectation that D4 would be prevalent
Debenedetti et al. (2024)	Agent attacks and defenses	AgentDojo: 97 tasks, 629 security tests	Dynamic benchmark	Defenses lowered but did not remove attack success	RQ3	Controls coded as breached or held, not only present
Hou et al. (2025)	Model Context Protocol	MCP server ecosystem and life cycle	Landscape review; threat taxonomy	Risks arise at creation, operation, and update of servers	RQ3	Ecosystem-component covariate; control C09
Jimenez et al. (2024)	LLM agents on real software tasks	SWE-bench: 2,294 GitHub issues	Benchmark of transformer LLMs (DL)	Agents can resolve real issues, with limited success	Scope	Defined agents that read, write, and run code as in scope
Yang et al. (2024)	Agent-computer interfaces	SWE-bench tasks	Agentic system design and ablation	Interface design strongly affects agent capability	RQ1	Autonomy features mapped to D1 and D8 controls
Chan et al. (2023)	Harms of agentic systems	Conceptual analysis	Sociotechnical framework	Agency blurs accountability and raises systemic harms	RQ1	Basis for domains D1 and D7
Shavit et al. (2023)	Governing agentic AI	Practitioner white paper	Normative practice list	Approval, legibility, attributability, interruptibility	RQ1, RQ3	Controls C02, C06, C15, C18
NIST (2023); OWASP Gen AI Security Project (2025)	AI risk governance	Consensus standards and threat lists	Expert consensus	Name functions and risks but do not test them on outcomes	RQ1, RQ3	20-control crosswalk; GCS makes controls testable
Allodi and Massacci (2014)	Vulnerability severity	NVD, exploit kits, attack data	Case-control study	CVSS is a weak proxy for exploitation	RQ3	CVSS read as technical impact; score-vector audit
McGregor (2021)	AI incident cataloguing	AI Incident Database	Crowdsourced incident records	Catalogues prevent repeats but vary in detail	Data	Chose standardized advisories as evidence base
Zheng et al. (2023)	LLM-as-a-judge	MT-Bench; Chatbot Arena	Agreement with human judgments	Strong LLM judges matched human-human agreement	RQ4	Human-human agreement used as the benchmark
Gilardi et al. (2023)	LLM text annotation	Tweets and news articles, four tasks	Zero-shot ChatGPT vs. crowd workers	LLM beat crowd workers on accuracy and intercoder agreement	RQ4	Justified a blinded, codebook-verbatim LLM coder

Author (Year)	Domain / Context	Dataset / Setting	Method(s)	Key Findings	RQ Linkage	How It Supports Project Decisions
Cemri et al. (2025)	Multi-agent LLM failures	Annotated agent execution traces	Grounded theory; LLM annotator	14 failure modes; LLM annotator close to experts	RQ4	Codebook-plus-LLM annotation design
Grinsztajn et al. (2022)	Tabular machine learning	45 medium-sized tabular data sets	Benchmark: trees vs. deep learning	Tree ensembles beat deep nets on tabular data	RQ3	Random forest and boosting as main learners; MLP as comparator
LeCun et al. (2015)	Deep learning	Review of representation learning	Narrative review	Multilayer networks learn features from data	RQ3, RQ4	Basis for the MLP model and the transformer LLM coder

Note. DL = deep learning; NVD = National Vulnerability Database; D1 to D8 and C01 to C20 are ACEG domains and controls (Appendix B).

Security of what assistants produce. The first body of work examines the code that assistants write. Pearce et al. (2022) found weaknesses in roughly two of every five Copilot-generated programs, and Perry et al. (2023) showed in a controlled experiment that developers using an assistant wrote less secure code while feeling more confident about it. These studies examine outputs, not actions. They do not cover an agent that runs a command, edits a configuration file, or calls a tool on its own initiative. That gap explains why ACEG gives four of its eight domains to agency, boundaries, oversight, and supply-chain integrity rather than to code quality, and why the perceived safety of a human checkpoint is tested in RQ2 rather than assumed.

Attacks on agents and the tool ecosystem. A second body of work shows that the inputs an agent reads can take over its goals. Greshake et al. (2023) described indirect prompt injection, in which instructions hidden in retrieved content redirect an LLM application without any action by the user. InjecAgent and AgentDojo then measured how widely tool-using agents fall for such attacks and how little current defenses help (Debenedetti et al., 2024; Zhan et al., 2024). Hou et al. (2025) extended the threat model to MCP servers, which are now a standard way to give coding agents new tools. Benchmarks control the task, but they cannot show how often each weakness reaches shipped products. The advisory census in this study supplies that field prevalence (RQ1), and component type enters RQ3 as a

covariate because many advisories concern extensions and servers rather than the agents themselves.

Governance without measurement. Chan et al. (2023) argued that agency, meaning goal-directedness, autonomy, and long-horizon action, spreads responsibility thinly and raises systemic harms. Shavit et al. (2023) proposed concrete practices such as constraining the action space, requiring approval, keeping actions legible, and making them attributable and interruptible. The NIST AI RMF, ISO/IEC 42001, and the OWASP agentic list turn similar ideas into functions and risk catalogues. What none of these sources provides is evidence that compliance with the controls relates to the seriousness of real failures. ACEG combines them into eight domains and 20 controls and adds the GCS, so the link between compliance and severity becomes a testable hypothesis (RQ3).

Evidence quality and automated coding. Incident catalogues such as the AI Incident Database show the value of collecting failures but record them unevenly (McGregor, 2021). Security advisories are more standardized and carry a CVSS rating. That rating measures technical impact rather than exploitation risk (Allodi & Massacci, 2014), so this study treats CVSS as an imperfect outcome and audits it. Coding hundreds of advisories by hand is slow, which is why recent work on LLM annotators matters. Gilardi et al. (2023) found zero-shot ChatGPT more accurate and more consistent than crowd workers on text-classification tasks. Zheng et al. (2023) reported that strong LLM judges agree with humans about as often as humans agree with each other, and Cemri et al. (2025) used an LLM annotator to apply a failure taxonomy to agent traces. These agentic and LLM studies shaped RQ4. The LLM coder receives the codebook verbatim and never sees severity, and it is judged against human agreement with a chance-corrected statistic (Cohen, 1960; Landis & Koch, 1977).

Model choice for small tabular data. Deep learning (DL) learns representations through many layers of nonlinear transformation (LeCun et al., 2015) and underlies both the

transformer models that power coding agents (Vaswani et al., 2017) and the LLM coder used here. For small structured data sets like this one, however, Grinsztajn et al. (2022) showed across 45 benchmarks that tree ensembles such as random forests (Breiman, 2001) and gradient boosting (Friedman, 2001) usually beat deep networks. The study therefore treats tree ensembles as its main learners, includes an MLP trained by backpropagation (Rumelhart et al., 1986) as the DL comparator, and keeps logistic regression as the interpretable model for the hypothesis test (Hosmer et al., 2013).

Gap addressed. Taken together, the literature describes coding-agent risks in the laboratory and recommends governance practices in principle. It does not score disclosed field weaknesses against an explicit control framework, test whether control failures predict severity, or check whether an LLM can apply such a framework reliably. This study addresses all three gaps.

Materials and Method

Data Source and Acquisition

The study uses two public sources: the GitHub Advisory Database (GitHub, 2026; https://github.com/github/advisory-database; Creative Commons Attribution 4.0) and the CVE List V5 maintained by the CVE Program (CVE Program, 2026; https://github.com/CVEProject/cvelistV5). Both are Git repositories of JavaScript Object Notation (JSON) records. They were acquired by cloning them at commits dated October 11, 2026, and the commit hashes are recorded in docs/data_freeze.md, so no web scraping, account, or application programming interface (API) key was needed. The script src/01_collect.py searched both clones for records published from January 1, 2023, that name a coding agent or one of its components (for example, Claude Code, Cursor, Copilot agent features, Codex CLI, Cline, Roo Code, Kiro, Aider, OpenHands, or MCP servers built for

these agents). It found 239 candidate records. The script src/02_screen.py merged records that appear in both sources on their CVE identifier and applied the eligibility rules.

Inclusion and Exclusion Criteria

A record was *included* if it disclosed a weakness in (a) an AI coding agent able to read, write, or execute code, commands, or tool calls, or (b) a component such an agent runs or that wraps it, such as an extension, plugin, hook, MCP server, bridge, build workflow, or distribution pipeline; it also had to be published between January 1, 2023, and October 11, 2026. A record was *excluded* for one of three reasons: E1, the product is not a coding agent or one of its components (14 records, for example browser assistants and name collisions); E2, the product only offers autocompletion with no ability to act (one record); or E3, the record has too little detail to code any domain (one record). The screening followed the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) flow (Page et al., 2021): 239 identified, 16 excluded, and 223 included. Because these are all the eligible records in the period, the study analyzes a census rather than a sample.

Data Description and Data Dictionary

The analysis file has one row per vulnerability: 223 cases and 21 source variables, to which coding adds the domain, control, and score fields. Cases run from October 31, 2023, to October 10, 2026. Of these, 155 were published in 2026, 66 in 2025, and one each in 2023 and 2024. The cases cover 20 products: 176 records concern agent products and 47 concern ecosystem components. The dependent variable, high_impact, equals 1 when the published CVSS severity is High or Critical (base score ≥ 7.0). That applies to 156 cases (70.0%): 126 High, 30 Critical, 49 Medium, and 18 Low. Table A1 in Appendix A is the full data dictionary, giving each variable's definition, type, allowed values, missing-value treatment, and analytic role.

Database Setup and Data Access

All analytic data live in one SQLite database (data/aceg.db; SQLite 3.53), created by src/08_build_gold_and_db.py. SQLite was chosen because it is a serverless, single-file relational store that any reviewer can open without installing a server, while still enforcing a schema and supporting Structured Query Language (SQL) joins and views. The database holds six tables and two views. The *cases* table holds the 223 source records, with a unique index on case_id. The tables *codes_coder1*, *codes_coder2*, *codes_llm*, and *codes_gold* hold one row per case for each label source, each with the same 30 coded fields and the derived scores. The *cleaning_log* table records every correction made to the coders' sheets. The view *v_analysis* joins the cases to the final labels and is the only input to the analysis. The view *v_agreement* lines up the three coders for reliability checks. The analysis script reads the data with pandas through Python's sqlite3 driver, for example SELECT * FROM v_analysis ORDER BY case_id, so every result traces back to a documented query. The completed coding workbooks are kept unchanged in coding/completed/ as the system of record.

Coding Procedure and Label Construction

Every case was coded against ACEG Version 1 (Appendix B). Coding a case means marking each of the eight domains present or absent, choosing one primary domain, coding each control of a present domain as breached (1) or not breached (0), and judging whether a breached control would have prevented the weakness (covered). Two human coders worked independently in separate copies of the coding workbook. Their worksheets showed only the case identifier, product, title, description, and link. Severity fields were withheld so that control judgments could not be influenced by the outcome.

The automated coder was Claude Opus 5.5, Anthropic's LLM, acting as an agentic annotator. It received the codebook rules and each case's title and description, with any severity wording masked by a regular expression (data/llm/blinded_case_text.txt). It returned

a compact record per case: domains present, primary domain, breached controls, and coverage. Controls of present domains that were not listed as breached were coded 0, as the codebook requires. The LLM coder never saw the human labels, the CVSS fields, or the outcome. The coding was done in one interactive session, not through the batch API script (src/03_llm_code.py), and Appendix F describes the ACEG Governance Skill built from the same coding procedure.

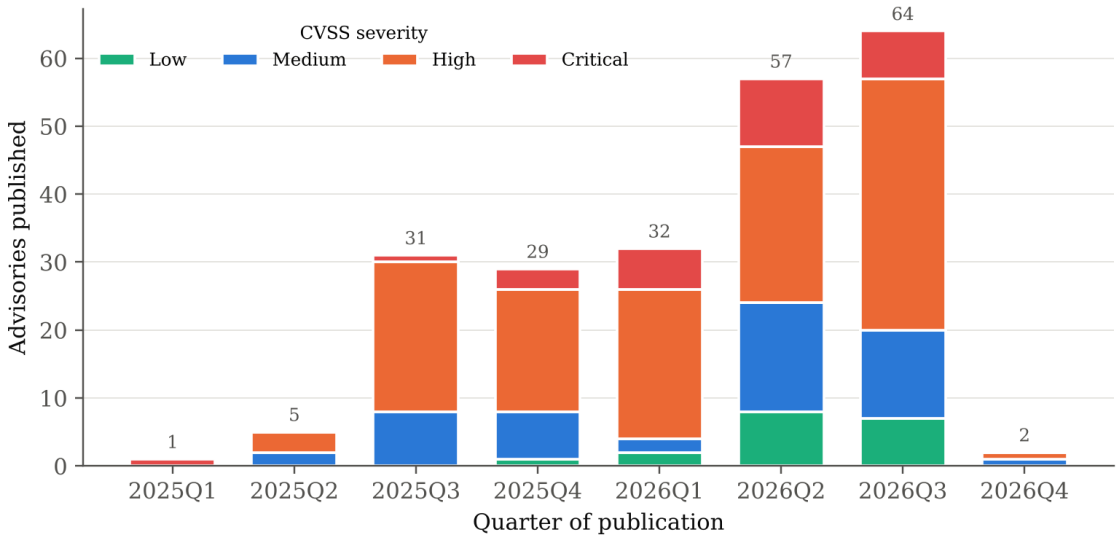
Two cleaning rules were applied to the human sheets before the labels were combined. First, Coder 1 had entered values for 171 control cells that belonged to domains the same coder had marked absent; under codebook rule 4 these were reset to not applicable, and each change is logged. Second, all values were normalized to 1, 0, or not applicable. Final labels were then formed cell by cell as the majority of the three coders. A domain is present when at least two coders coded it present, and a control is breached when most of the coders who judged it coded it breached. Seven cases in which no domain reached two votes, and 51 primary-domain ties, were resolved with pre-stated tie-break rules: most votes first, then the earliest point in the causal chain. The rules are documented in the script header. Because the final labels include the LLM's vote, RQ4 does not use them. It scores the LLM only against cases on which the two humans agreed.

Exploratory Data Analysis

The exploratory data analysis (EDA) asked four questions that shaped the models: how the cases are spread over time, how the outcome is distributed, whether cases cluster by product, and which characteristics are associated with high impact. Figure 1 shows that disclosures rose from a handful a quarter before mid-2025 to 64 in the third quarter of 2026, while the High and Critical share stayed high throughout. The outcome is therefore not an artefact of one period, and because 2023 and 2024 have only one case each, time enters the models as a single year_2026 indicator.

Figure 1

Coding-Agent Advisories by Quarter of Publication and CVSS Severity



Note. Quarters from 2025 onward are shown. The two earlier cases (2023 and 2024) and the partial fourth quarter of 2026 (data frozen October 11, 2026) are included in all analyses.

Base scores cluster between 7.0 and 9.0 (median 7.8 before recovery; Figure C1). The 70:30 class split means a model that always predicts high impact is already about 70% accurate, so models are judged on AUC, balanced accuracy, and the Brier score (Brier, 1950) rather than raw accuracy. Three products, Claude Code (58 cases), Cursor (32), and GitHub Copilot (24), account for 51% of the cases, so the RQ3 logistic model is re-estimated with standard errors (*SEs*) clustered by product. Table 2 tests each characteristic against the outcome. Weakness type is strongly associated with high impact (Cramér's V = 0.43). Figure 2 shows that every code-injection case and 88% of command-injection cases are high-impact, compared with 29% of information-exposure cases. Component type, publication year, and user interaction show no association. The CVSS version is associated with the outcome, so it is tested as a sensitivity covariate. The attack vector is never used as a predictor because it is part of the CVSS formula.

Table 2

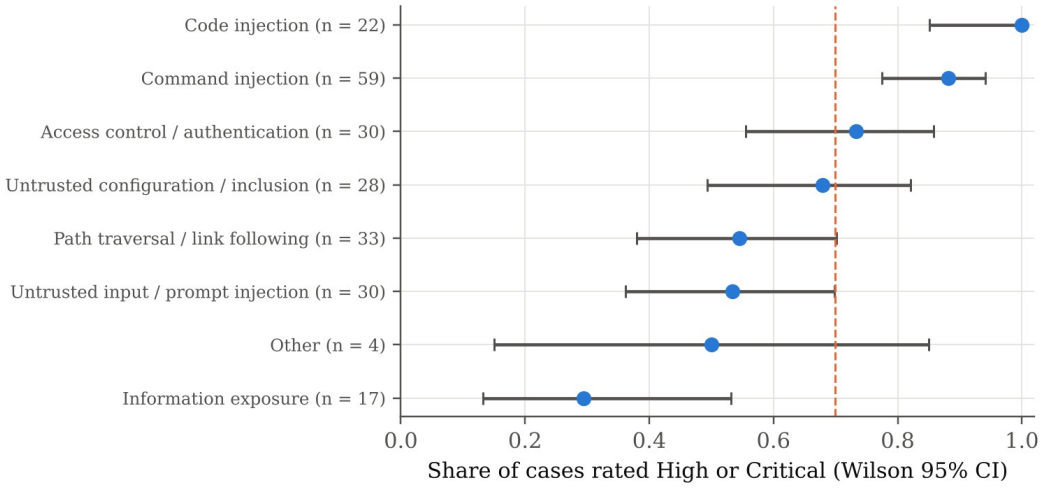
Association of Case Characteristics With High Impact

Characteristic	Levels	χ^2	*df*	*p*	Cramér's V	*n*	Use in modelling
CWE group	8	40.67	7	< .001	.43	223	Context; M3, M7 features
Ecosystem component	2	1.06	1	.302	.07	223	Covariate
Published in 2026	2	0.59	1	.441	.05	223	Covariate
User interaction required	2	1.26	1	.262	.08	221	RQ2 variable only
CVSS version 4.0	2	5.86	1	.015	.16	223	Sensitivity covariate

Note. Pearson chi-square tests without continuity correction. The CWE-group table has expected counts below 5 in some cells, so it is used descriptively.

Figure 2

High-Impact Rate by Weakness (CWE) Group



Note. Points are proportions rated High or Critical; bars are Wilson 95% CIs; the dashed line is the overall rate (70%).

The final labels add a governance view (Figure 7 in Results). Agency and action control (D1) and input trust and goal integrity (D4) dominate and appear together in 74 cases: untrusted content leads an agent to run a command. Transparency (D3) and accountability (D7) are rare. The GCS is concentrated between 50% and 60% (median 57.1%) and is left-skewed (skewness = −2.35), because most cases breach two to four of five to ten applicable controls. One more EDA finding mattered for RQ3: the share of high-impact cases rises from 41% among single-domain cases to 78% among two-domain cases and to 85% or more

among cases with four or more domains. This motivated keeping the number of domains as a covariate.

Research Hypotheses

Each research question is tested with a pair of hypotheses at the 5% significance level ($\alpha = .05$). Table 3 states them, along with the statistical test and the decision rule.

Table 3

Research Hypotheses, Tests, and Decision Rules

RQ	Null hypothesis (H_0)	Alternative hypothesis (H_a)	Statistical test	Decision at $\alpha = .05$
RQ1	$H1_0$: $\pi \leq .80$ (ACEG covers at most 80% of cases)	$H1_a$: $\pi > .80$	Exact one-sided binomial test; Wilson 95% CI	Reject $H1_0$ if $p < .05$
RQ2	$H2_0$: domain family and user interaction are independent	$H2_a$: they are associated	Pearson χ^2 test of independence (4 × 2); Cramér's V; Monte Carlo p	Reject $H2_0$ if $p < .05$
RQ3	$H3_0$: $\beta_{GCS} = 0$ (OR = 1) in the adjusted logistic model	$H3_a$: $\beta_{GCS} \neq 0$ (OR < 1 expected)	Binary logistic regression; Wald z and likelihood-ratio tests	Reject $H3_0$ if $p < .05$
RQ4	$H4_0$: $\kappa \leq .60$ (LLM vs. human consensus)	$H4_a$: $\kappa > .60$	Cohen's κ with case-level bootstrap CI (2,000 resamples)	Reject $H4_0$ if lower 95% bound > .60

Minimum Sample Size

Each research question has its own sample-size calculation matched to its analysis, all with $\alpha = .05$ (two-sided) and power = .80 (Table 4). RQ1 estimates a proportion, so the confidence-interval method with the most conservative $p = .50$ and a margin of ±.07 is used. RQ2 tests association in a 4 × 2 table ($df = 3$), so the noncentral chi-square power method is used with Cohen's (1988) effect size w = 0.25, between small and medium. RQ3 tests a continuous predictor in logistic regression, so the method of Hsieh et al. (1998) is used with the observed high-impact proportion P = .70 and a target OR of 0.61 per standard deviation (*SD*), inflated for the variance the covariates share with GCS ($R^2 = .20$). RQ4 estimates an agreement proportion with the confidence-interval method, using expected accuracy a = .80 (Zheng et al., 2023) and a margin of ±.06.

Table 4

Minimum Sample Size by Research Question

RQ	Formula	Parameters and computation	Minimum n
RQ1	$n = Z^2p(1 - p) / e^2$	Z = 1.96, p = .50, e = .07: 3.8416 × 0.25 / 0.0049 = 196.0	196
RQ2	$n = \lambda / w^2$	df = 3, χ^2_{crit} = 7.815, λ = 10.903 for power .80: 10.903 / 0.0625 = 174.4	175
RQ3	$n = (Z_{1-\alpha/2} + Z_{1-\beta})^2 / [P(1 - P)\beta^{*2}]$; $n_{adj} = n / (1 - R^2)$	(1.960 + 0.842)² / (0.21 × 0.25) = 149.5; 149.5 / 0.80 = 186.9	187
RQ4	$n = Z^2a(1 - a) / e^2$	a = .80, e = .06: 3.8416 × 0.16 / 0.0036 = 170.7	171

Note. e = margin of error; w = Cohen's effect size; λ = noncentrality parameter; P = proportion of high-impact cases (156 of 223); β* = |log odds ratio| per SD = 0.50 (OR ≈ 0.61); a = expected agreement. Reproduced by sample_size/sample_size.py.

The binding requirement is max(196, 175, 187, 171) = 196 cases. Only 223 eligible advisories exist, so all of them are analyzed and the collected volume exceeds the minimum by 14%. At N = 223 the RQ1 margin of error narrows to ±6.6 points. RQ2 has power 0.89 to detect w = 0.25 with the 221 cases that have a CVSS vector. The 67 low-impact cases give 16.8 events per predictor in the four-predictor RQ3 model, above the minimum of 10 (Peduzzi et al., 1996). For RQ4, the per-domain consensus sets contain 94 to 189 cases and the pooled set 1,214 domain cells, above the 171 required. The data volume is therefore sufficient for every research question, and the remaining limits are the specific cells reported with each result, not the overall sample.

Statistical Methods and Model Selection

Each method was chosen to fit the type of question. RQ1 estimates proportions, so it uses Wilson score intervals, which keep their nominal coverage near 0 and 1 (Wilson, 1927), and an exact binomial test. RQ2 crosses two categorical variables, so it uses Pearson's χ^2 test with Cramér's V (Cramér, 1946) and standardized residuals. Because some expected counts are below 5, a 10,000-draw Monte Carlo permutation test and a test with merged families are added. RQ3 has a binary outcome, so the hypothesis is tested with binary logistic regression: $\text{logit}(P[\text{high impact}]) = \beta_0 + \beta_1 GCS_z + \beta_2 \text{ecosystem} + \beta_3 \text{n_domains} + \beta_4 \text{year_2026}$. Logistic regression gives interpretable odds ratios and has well-understood diagnostics (Hosmer et al.,

2013). Its assumptions were checked with the Hosmer–Lemeshow test, the Box–Tidwell test for a linear logit (Box & Tidwell, 1962), variance inflation factors (VIFs), and *SEs* clustered by product.

Prediction is a separate goal from testing the hypothesis, so eight models were compared. M0 is a majority-class baseline. M1, M2, and M3 are logistic regressions on covariates only, on GCS plus covariates, and on CWE group, the last representing current practice, in which teams triage by weakness type. M4, M5, and M6 are a random forest (Breiman, 2001), histogram gradient boosting (Friedman, 2001), and an MLP neural network, all trained on the full ACEG profile: eight domains, 20 breach indicators, GCS, and covariates. M7 is a random forest on the ACEG profile plus CWE group. Tree ensembles were favoured because they capture interactions between controls without specification and perform best on small tabular data (Grinsztajn et al., 2022). The MLP was included as the DL comparator. RQ4 compares labels from different coders, so it uses Cohen's κ for pairs (Cohen, 1960), Fleiss' κ for three raters (Fleiss, 1971), and accuracy, precision, recall, and F1 for the LLM against the human consensus, with bootstrap CIs (Efron & Tibshirani, 1993). Agreement is read on the Landis and Koch (1977) scale, so $\kappa > .60$ means substantial agreement (McHugh, 2012).

Data Splitting and Validation

For prediction, the 223 cases were split once into a stratified training set of 167 cases (75%; 117 high-impact) and a test set of 56 cases (25%; 39 high-impact), using random seed 42. Stratification kept the 70:30 balance in both parts. The case-level split is saved in outputs/final/train_test_split.csv. All model selection used the training set only. Hyperparameters for M4 to M7 were tuned by grid search with stratified five-fold cross-validation (CV) on AUC, and each tuned model was then scored by stratified five-fold cross-validation repeated 10 times (50 folds) on the training set. The best model was chosen *before*

the test set was used, by the highest mean cross-validated AUC. The test set was used once, to report out-of-sample performance. As a robustness check, all eight models were also evaluated by repeated five-fold cross-validation on the full data set. The RQ3 hypothesis model, which is inferential rather than predictive, was fitted to all 223 cases.

Original Work and Comparison With Current Methods

This study's original contributions are as follows. It defines a governance framework for coding agents that can be tested against field evidence. It produces a three-coder data set of the full population of disclosed coding-agent weaknesses. It audits CVSS records against their own vectors. It formally compares predictors based on governance coding with current weakness-type triage. It measures an LLM coder against human agreement in a way that is not circular. Current practice triages advisories by CVSS score and CWE type, one record at a time. M3 represents that practice, so the gap between M3 and the ACEG models measures what governance coding adds.

Architecture Diagram/Workflow

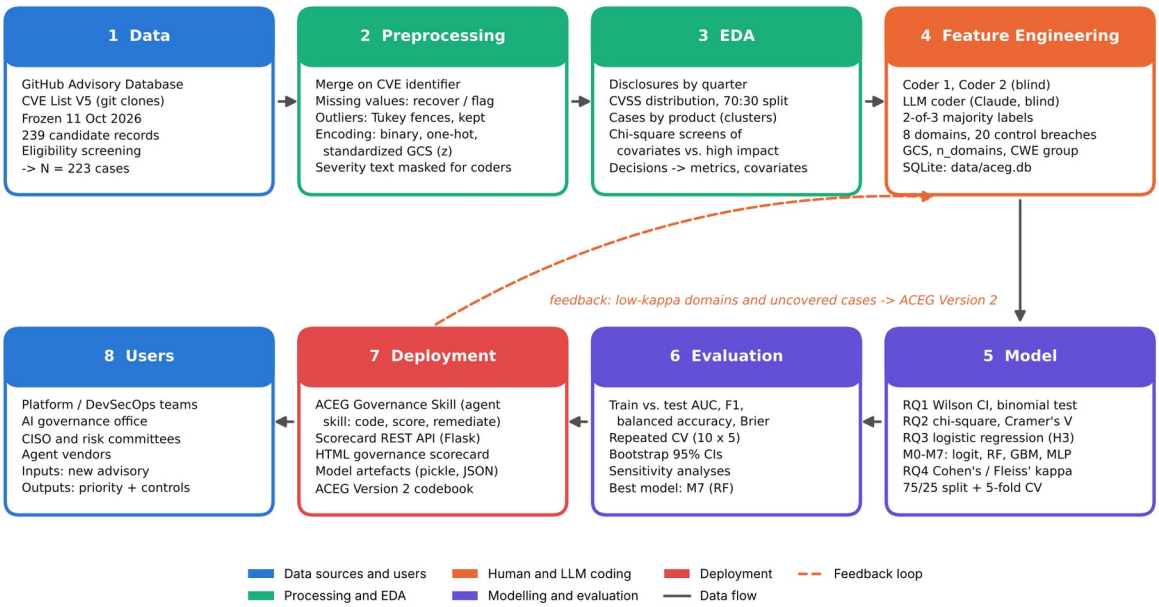
System Overview

The pipeline turns raw advisory records into three outputs: evidence on the research questions, a trained prediction model, and a scorecard that teams can use for new advisories. It runs in seven stages, from data acquisition to deployment, with a feedback loop from deployment back to the codebook. Each stage is a numbered Python script that reads from and writes to the repository or the SQLite database, so the whole workflow can be rerun from the frozen commits. Every stage runs on a standard central processing unit (CPU); no graphics processing unit (GPU) is required.

Architecture Diagram

Figure 3

End-to-End Workflow of the Proposed System



Note. Colours mark the type of stage (see legend). Solid arrows show data flow from the frozen sources to users; the dashed arrow is the feedback loop through which low-agreement domains and uncovered cases revise the codebook (ACEG Version 2).

As shown in Figure 3, data move from the two Git sources (stage 1) through preprocessing (stage 2) and EDA (stage 3) to feature engineering (stage 4), where the human and LLM coders turn advisory text into governance features stored in SQLite. The models (stage 5) and their evaluation (stage 6) feed the deployment layer (stage 7), which serves the users (stage 8). The feedback loop carries evidence from deployment, such as weak agreement and framework gaps, back into the codebook.

Workflow Components

Data Ingestion. The sources are two Git repositories of JSON advisory records, cloned at fixed commits. src/01_collect.py parses the JSON, filters by publication date and by agent and component names, and writes the 239 candidates to comma-separated values (CSV). src/02_screen.py merges duplicates on the CVE identifier, applies the exclusion

codes, and writes the 223 cases. The coders' Excel workbooks and the LLM output are ingested by src/07_parse_llm_codes.py and src/08_build_gold_and_db.py, which load them into SQLite.

Data Preprocessing. *Missing values* were handled according to why they were missing. Of the 31 cases without a numeric CVSS score, 24 were recomputed from their own version 3.x vectors with the FIRST (2019) formula. All 24 recovered scores match the published severity band. The remaining 7 cases (version 4.0 or no vector) were left missing because the score is not a model input and the outcome uses the published severity, which is complete. The 82 missing titles were filled with the first 120 characters of the description. The two cases without a CVSS vector were excluded only from RQ2, which needs that field. Identifier fields that do not exist in a source (77 GitHub-only and 4 CVE-only identifiers) were left structurally missing. *Outliers* were found with Tukey fences: three CVSS scores below 3.2 and 26 very long descriptions. They were kept because they are valid records rather than errors, and tree models and the bounded GCS are robust to them. *Encoding:* component type, year, and the CVSS version became 0/1 indicators; CWE identifiers were mapped to eight groups and one-hot encoded; control codes were recoded as breach indicators (1 = breached, otherwise 0). *Scaling:* GCS was standardized (*z*-score) for the logistic model so that its OR is per *SD*, and the MLP and logistic pipelines standardized all inputs with scikit-learn's StandardScaler, fitted on the training folds only. The tree models need no scaling. A consistency audit found that in 43 of the 132 cases with a version 3.x vector, the published score differs from the score the vector implies, because the GitHub advisory and the CVE record can carry ratings from different organizations. The severity band differs in 19 cases and the binary outcome in 6. The published severity was kept as the pre-registered outcome.

Exploratory Data Analysis (EDA). The EDA (Figures 1, 2, and C1; Table 2) produced descriptive statistics, the distribution of the outcome, product clustering, and χ^2 screens of each characteristic. Each finding was linked to a modelling decision: imbalance-robust metrics, a single year indicator, product-clustered *SEs*, and CVSS version as a sensitivity covariate.

Feature Engineering. Feature engineering turns text into governance features in three steps. First, three coders produce the domain, control, and coverage codes, which are combined into majority labels. Second, derived features are computed: the GCS = 100 × (applicable − breached) / applicable, the number of domains present, the domain family (A to D) of the primary domain, and 20 breach indicators. Third, covariates and CWE-group dummies are added. CVSS fields are never used as features, to avoid leakage, because they define the outcome.

Model Development. The models are the eight described under Materials and Method, from M0 to M7. They were trained on the 75% training split, with hyperparameters tuned by stratified five-fold grid search. The selected settings were: M4 random forest, 500 trees, minimum leaf size 5, √p features per split, balanced class weights; M5 gradient boosting, learning rate 0.03, depth 3, 100 iterations; M6 MLP, two hidden layers of 32 and 16 rectified linear units, L2 penalty 0.001; M7 random forest, 500 trees, maximum depth 5, minimum leaf size 3.

Model Evaluation. Every model was scored on the training set, by repeated cross-validation within the training set, and on the untouched test set. The metrics were AUC with a bootstrap CI, accuracy, balanced accuracy, precision, recall, F1 at a threshold of 0.5, and the Brier score. A paired bootstrap compared AUCs, and permutation importance explained the best model.

Deployment. The selected model, refitted on all 223 cases, is saved as a model file (models/aceg_risk_model.pkl). It is served by a Flask representational state transfer (REST) API (src/10_scorecard_service.py) with POST /score and GET /scorecard/<case_id> endpoints, and it renders a hypertext markup language (HTML) governance scorecard. The same model, with a dependency-free fallback, is packaged in the ACEG Governance Skill (skill/aceg-governance/; Appendix F), which carries the codebook, blinding, scoring, and remediation steps into any LLM agent environment.

Tools and Technologies

Table 5 lists the languages, libraries, platforms, and hardware used at each stage. All software is open source except the LLM coder, and the package versions are pinned in requirements.txt so that the results can be reproduced.

Table 5

Tools and Technologies Used

Component	Tools, libraries, and platform
Language and runtime	Python 3.13.18; Node.js (rendering only)
Data handling	pandas 3.0.5, NumPy 2.5.3, openpyxl 3.1.5
Statistics	SciPy 1.18.1 (χ^2, binomial, Mann–Whitney); custom NumPy maximum-likelihood logistic regression with clustered *SEs* (src/stats_utils.py)
Machine learning and DL	scikit-learn 1.9.1 (Pedregosa et al., 2011; LogisticRegression, RandomForestClassifier, HistGradientBoostingClassifier, MLPClassifier, GridSearchCV, permutation_importance)
LLM / agentic coding	Claude Opus 5.5 (Anthropic) as blinded coder; Anthropic API pipeline script for batch reruns
Database	SQLite 3.53 via Python sqlite3 (data/aceg.db)
Visualization	Matplotlib 3.11.2; Playwright with Chromium for scorecard screenshots
Deployment	ACEG Governance Skill (Agent Skill format: SKILL.md, references, scripts, assets); Flask 3.1.3 REST API; pickle and JSON model artefacts; HTML scorecard
Compute	Cloud Linux container with a 2-core Intel Xeon central processing unit (CPU) at 2.10 GHz and 7 GB memory; no graphics processing unit (GPU) used (full pipeline about 4 minutes)
Version control	Git and GitHub (public repository)

Results

This section reports the predictive model comparison first, followed by the visual evidence and the answer to each research question.

Model Performance

Table 6

Model Performance Comparison (Training, Cross-Validation, and Hold-Out Test)

Model	Features used	AUC train / CV / test	Accuracy train / test	Bal. acc. train / test	Test F1	Test Brier	Key observation
M0 Majority class	None (class prior)	.50 / .50 / .50	.70 / .70	.50 / .50	.82	.211	Floor: 70% accuracy from imbalance alone
M1 Logistic (covariates)	Component type, number of domains, 2026	.73 / .71 / .67	.74 / .71	.62 / .56	.82	.192	Covariates carry modest signal
M2 Logistic (GCS + covariates)	GCS + M1 covariates	.75 / .72 / .69	.75 / .71	.63 / .55	.83	.188	GCS adds little over covariates
M3 Logistic (CWE group)	CWE-group indicators, component type, 2026	.80 / .74 / .70	.78 / .68	.70 / .55	.79	.197	Current triage; better than GCS but well below ACEG
M4 Random forest (ACEG)	8 domains, 20 control breaches, GCS, covariates	.89 / .83 / .84	.81 / .75	.82 / .70	.82	.162	ACEG profile alone lifts AUC by about .14 over M3
M5 Gradient boosting (ACEG)	As M4	.92 / .82 / .84	.81 / .73	.83 / .69	.81	.148	Similar to M4; best calibrated (Brier)
M6 MLP neural network (ACEG)	As M4 (standardized)	.97 / .78 / .85	.90 / .77	.86 / .67	.85	.155	Overfits (train AUC .97); highest test AUC by chance
M7 Random forest (ACEG + CWE)	**M4 features + CWE groups**	**.93 / .85 / .84**	**.84 / .73**	**.86 / .68**	**.81**	**.155**	**Best: highest CV AUC; selected before testing**

Note. AUC train / CV / test: training-set AUC, mean AUC of stratified 5-fold cross-validation repeated 10 times within the training set, and hold-out test AUC (bootstrap 95% CIs: M2 [.52, .86]; M3 [.54, .85]; M4 [.73, .94]; M5 [.71, .95]; M6 [.70, .96]; M7 [.73, .94]). Validation: stratified 75/25 hold-out split (167 training, 56 test cases). Accuracy, balanced accuracy, and F1 use a 0.5 threshold. The best model (shaded) was chosen on CV AUC before the test set was scored. Full-data repeated CV AUCs (Table D2): M7 = .842, M4 = .829, M5 = .810, M6 = .790, M3 = .736, M2 = .712.

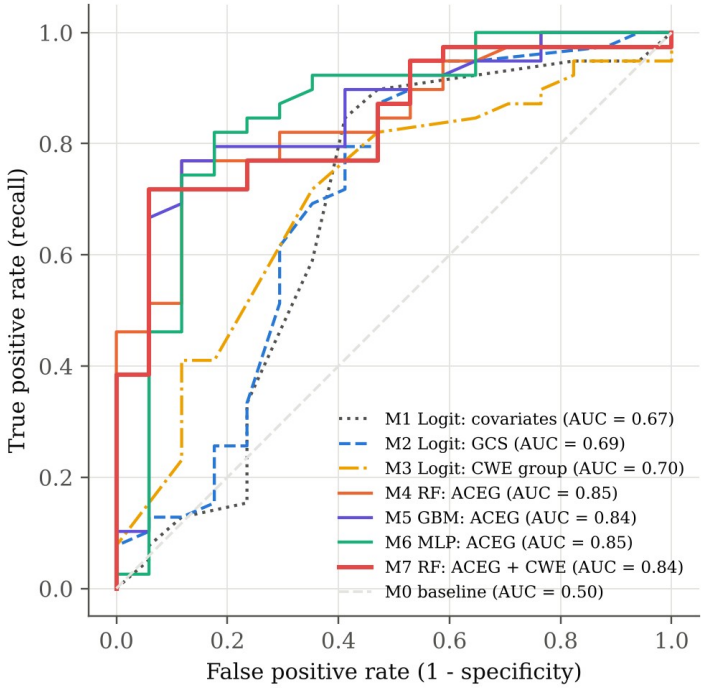
Table 6 compares the eight models. The random forest on the ACEG profile plus weakness type (M7) is the best model. It had the highest cross-validated AUC on the training data (.85, *SD* = .06), which was the selection criterion fixed in advance, and the highest AUC in full-data repeated cross-validation (.842). On the 56-case test set it reached AUC = .84, 95% CI [.73, .94], balanced accuracy = .68, F1 = .81, and Brier = .155. Its training AUC (.93) is higher than its test AUC, a normal gap for a random forest that stays within the cross-validation spread. The three tree-based ACEG models (M4, M5, and M7) form a group, with test AUCs between .840 and .845 that cannot be told apart on 56 cases. The MLP's test AUC (.85) is just as high, but its training AUC of .97 and lower cross-validated AUC (.78) show overfitting, as Grinsztajn et al. (2022) would predict. The improvement over the GCS model

and over current triage is the substantive result: a paired bootstrap gives M7 − M2 = +0.14 AUC (95% CI [-0.00, 0.30], one-sided p = .025) and M7 − M3 = +0.14 (one-sided p = .058), and in full-data cross-validation the ACEG forest beats weakness-type triage by about .11 AUC.

Visual Evidence

Figure 4

Receiver Operating Characteristic Curves on the Hold-Out Test Set

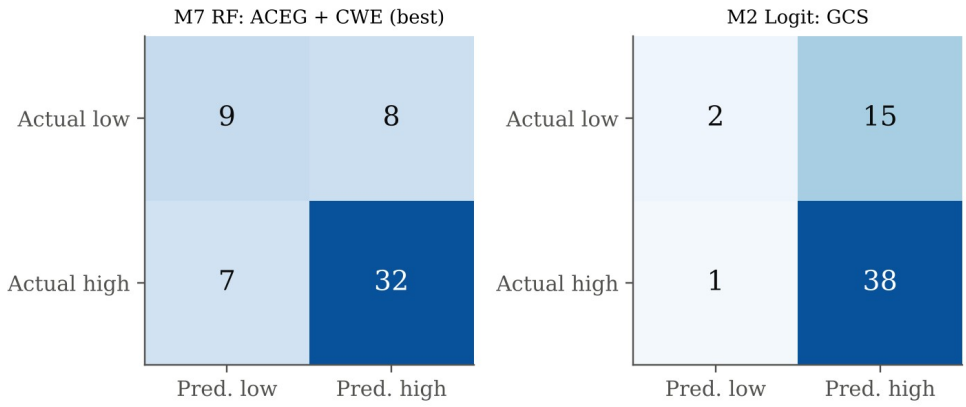


Note. n = 56 test cases (39 high-impact). Logit = logistic regression; RF = random forest; GBM = gradient boosting; MLP = multilayer perceptron. The dashed diagonal is the majority-class baseline.

Figure 4 shows why the ACEG tree models are preferred. Their ROC curves rise steeply at low false-positive rates: at about 10% false positives, M7 already finds roughly 70% of the high-impact cases. The logistic GCS and covariate models only separate the classes at much higher false-positive rates. This matters operationally, because a triage list is useful only if its top entries are mostly true positives.

Figure 5

Confusion Matrices for the Best Model (M7) and the GCS Logistic Model (M2) on the Test Set

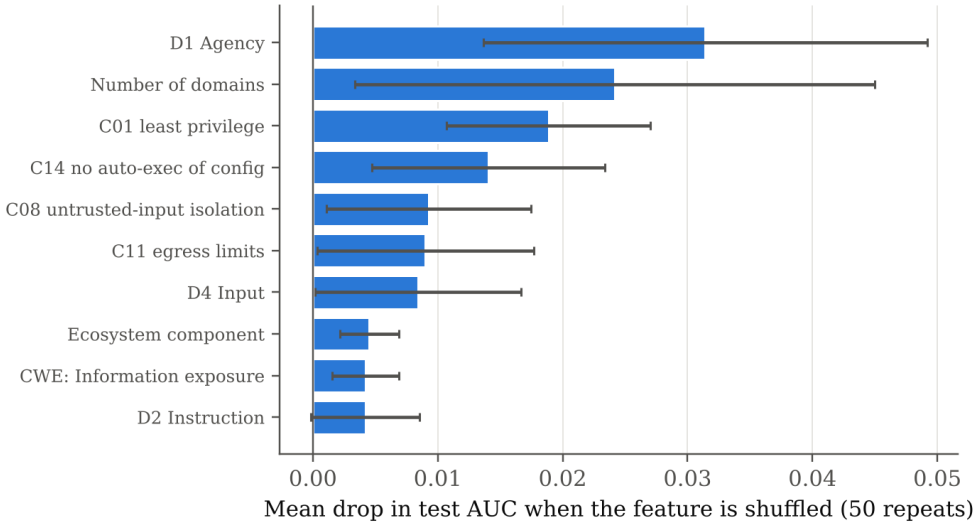


Note. Threshold = 0.5. Rows are the actual class; columns are the predicted class.

Figure 5 makes the difference concrete. M2 labels almost every case high-impact. It catches 38 of 39 high-impact cases but clears only 2 of 17 low-impact ones (specificity = .12), so it is no better than CVSS-free guessing at the 70% base rate. M7 correctly identifies 9 of 17 low-impact cases and 32 of 39 high-impact cases (specificity = .53, recall = .82), so it can tell triage teams which advisories can safely wait.

Figure 6

Permutation Importance of the Ten Most Important Features in the Best Model (M7)



Note. Importance is the mean drop in test-set AUC when a feature is randomly shuffled (50 repeats); bars show ±1 *SD*.

Figure 6 shows that the model relies on governance features rather than weakness type. The most important features are the presence of D1 (agency and action control), the number of domains, a breached C01 (least privilege), a breached C14 (no auto-execution of untrusted workspace configuration), and breached C08 and C11 (untrusted-input isolation and egress limits). Only one CWE group (information exposure, which lowers risk) makes the top ten. In practical terms, a weakness is likely to be severe when the agent can act beyond its authority, runs with broad privileges, or executes repository configuration automatically.

Results by Research Question

RQ1: Framework Coverage and Domain Prevalence

The framework covered 222 of 223 cases (99.6%, Wilson 95% CI [97.5, 99.9]). The exact one-sided binomial test rejects $H1_0$ ($\pi \leq .80$), $p < .001$, so ACEG covers more than 80% of disclosed coding-agent weaknesses. The result does not depend on the label source. Each coder's own coverage also exceeds 80%: 100% for Coder 1, 99.1% for the LLM, and 83.0%

for Coder 2, although the last is not significant on its own (p = .153). The one uncovered case, CA-121, is a secure shell (SSH) host-key verification flaw in a desktop client. It is a conventional transport-security weakness for which ACEG has no control, and it is logged for Version 2. Table 7 and Figure 7 give the prevalence of each domain.

Table 7

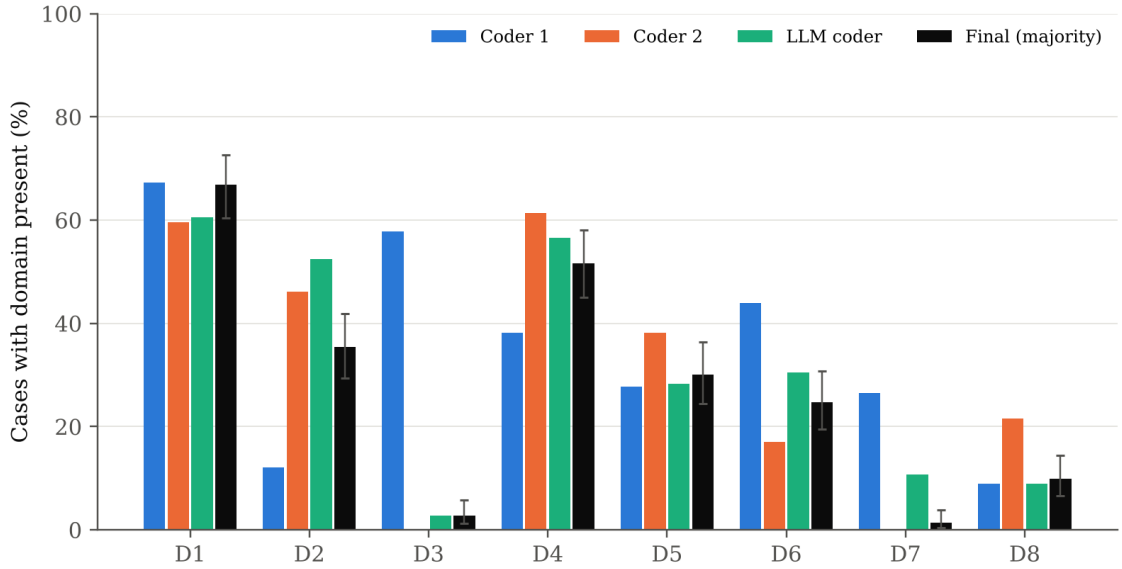
Prevalence of the Eight ACEG Governance Domains (N = 223)

Domain	Family	Final n (%)	Wilson 95% CI	Coder 1 %	Coder 2 %	LLM %
D1 Agency and action control	A	149 (66.8)	[60.4, 72.7]	67.3	59.6	60.5
D2 Instruction and boundary adherence	A	79 (35.4)	[29.4, 41.9]	12.1	46.2	52.5
D3 Transparency and honest reporting	D	6 (2.7)	[1.2, 5.7]	57.8	0.0	2.7
D4 Input trust and goal integrity	B	115 (51.6)	[45.0, 58.0]	38.1	61.4	56.5
D5 Secrets and data protection	C	67 (30.0)	[24.4, 36.4]	27.8	38.1	28.3
D6 Code and supply-chain integrity	B	55 (24.7)	[19.5, 30.7]	43.9	17.0	30.5
D7 Accountability and recoverability	D	3 (1.3)	[0.5, 3.9]	26.5	0.0	10.8
D8 Human oversight and reliance	A	22 (9.9)	[6.6, 14.5]	9.0	21.5	9.0

Note. Families: A = agency and oversight; B = input and code integrity; C = data protection; D = transparency and accountability. A case may have several domains (final-label mean = 2.22).

Figure 7

Domain Prevalence by Label Source



Note. Error bars on the final-label bars are Wilson 95% CIs. The final label is the 2-of-3 majority of Coder 1, Coder 2, and the LLM coder.

Agency and action control (D1, 66.8%) and input trust and goal integrity (D4, 51.6%) dominate, followed by boundary adherence (D2, 35.4%), secrets and data protection (D5, 30.0%), and code and supply-chain integrity (D6, 24.7%). Human oversight (D8, 9.9%), transparency (D3, 2.7%), and accountability (D7, 1.3%) are rare. Among controls, C08 (isolation of untrusted inputs) is breached in 111 cases, 96.5% of those where it applies. It is followed by C02 (approval gate, 72 cases), C04 (machine-enforced policy limits, 65), C01 (least privilege, 64), and C14 (no auto-execution of workspace configuration, 45). The typical failure chain is that untrusted content enters the context, the agent bypasses an approval or allowlist, and a command runs with the developer's privileges.

RQ2: Domain Family and Need for Human Interaction

Table 8

Domain Family of the Primary Domain by User Interaction Required (n = 221)

Domain family	No interaction (UI:N)	Interaction required	Total	% interaction	Std. residual (UI:N)
A Agency and oversight	40	53	93	57.0	-0.06
B Input and code integrity	47	63	110	57.3	-0.11
C Data protection	9	8	17	47.1	0.59
D Transparency and accountability	0	1	1	100.0	-0.66

Note. $\chi^2(3) = 1.42$, $p = .700$, Cramér's V = .08; Monte Carlo $p = .827$ (10,000 permutations); with family D merged into A, $\chi^2(2) = 0.68$, $p = .713$. Two cases lack a CVSS vector.

Table 8 shows nearly identical interaction rates in the three populated families: 57.0% for agency and oversight, 57.3% for input and code integrity, and 47.1% for data protection. The association is not significant, $\chi^2(3) = 1.42$, $p = .700$, V = .08, and the Monte Carlo test ($p = .827$) and the merged-family test ($p = .713$) agree, so $H2_0$ is retained. With power .89 to detect a small-to-medium effect, this is informative: weaknesses in every family are about equally likely to need, or to bypass, a human action. A human checkpoint does not protect more against one kind of failure than another. Approval prompts cannot stand in for the domain-specific controls, which is consistent with the finding of Perry et al. (2023) that

people over-trust assistant output. The result held with each coder's own labels (all $p \geq .12$; Table D4).

RQ3: Governance Compliance and High Impact

Table 9

Logistic Regression Predicting a High-Impact Case (Hypothesis Model, N = 223)

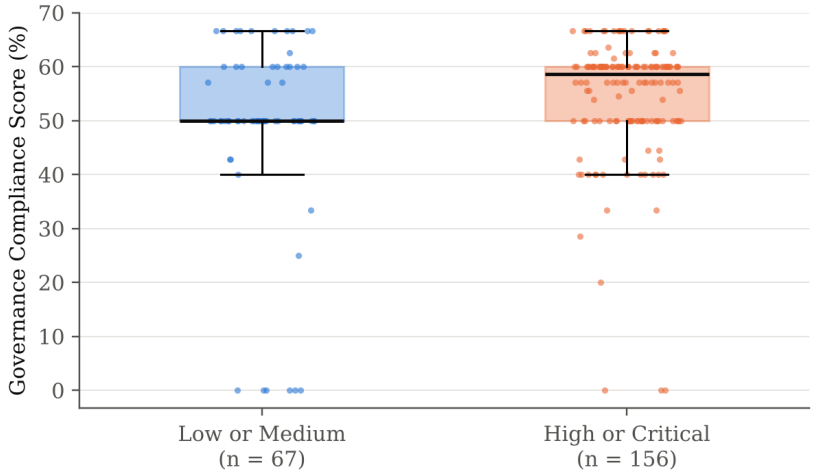
Term	β	*SE*	*z*	*p*	OR	95% CI for OR	Product-clustered 95% CI
Intercept	-0.723	0.460	-1.57	.116	0.49	[0.20, 1.19]	[0.16, 1.50]
GCS (per *SD*)	0.293	0.154	1.90	.057	1.34	[0.99, 1.81]	[0.99, 1.82]
Ecosystem component	-0.071	0.398	-0.18	.859	0.93	[0.43, 2.03]	[0.53, 1.62]
Number of domains	0.824	0.187	4.40	< .001	2.28	[1.58, 3.29]	[1.50, 3.45]
Published in 2026	-0.123	0.358	-0.35	.730	0.88	[0.44, 1.78]	[0.45, 1.75]

Note. Likelihood-ratio $\chi^2(4) = 31.88$, $p < .001$; McFadden $R^2 = .117$; Hosmer–Lemeshow $\chi^2(8) = 6.22$, $p = .623$; Box–Tidwell $p = .400$; VIFs 1.01 to 1.12. GCS is standardized (*SD* = 13.7 points).

Table 9 reports the hypothesis model. The model is significant overall (likelihood-ratio $\chi^2[4] = 31.88$, $p < .001$), fits adequately (Hosmer–Lemeshow $p = .623$), meets the linear-logit assumption (Box–Tidwell $p = .400$), and shows no multicollinearity (all VIFs < 1.2). The GCS odds ratio is 1.34 per *SD* (95% CI [0.99, 1.81], $p = .057$). The CI includes 1, so $H3_0$ is *not rejected* at $\alpha = .05$, and the point estimate is in the direction opposite to the expected OR < 1. In contrast, each additional governance domain implicated more than doubles the odds of a high-impact case (OR = 2.28, 95% CI [1.58, 3.29], $p < .001$), and this holds with product-clustered *SEs*.

Figure 8

Governance Compliance Score by Impact Class



Note. Boxes show quartiles; points are individual cases. Medians: 50.0% (low or medium) and 58.6% (high or critical); Mann–Whitney p = .009.

Figure 8 explains the result. High-impact cases have a slightly *higher* median GCS (58.6% vs. 50.0%), and the unadjusted difference is significant (Mann–Whitney p = .009). It reverses the expected direction because GCS is a ratio: a case that implicates more domains brings more applicable controls into the denominator, and many of them hold. The sensitivity analyses (Table D1; Figure D1) confirm that GCS is not a stable predictor. Its OR is about 1.1 and nonsignificant with the Coder 1 or LLM labels, 0.72 (p = .045) with the Coder 2 labels, and 1.12 (p = .784) when the raw count of breached controls replaces GCS, and adding a CVSS-version covariate changes nothing. The substantive answer to RQ3 is therefore about breadth, not depth: what predicts severity is *how many* governance domains fail together and *which* controls fail (Figure 6), not the share of controls that held. A single summary score hides that profile, which is why the profile-based models outperform the GCS model by a wide margin.

RQ4: Human Versus LLM Coding

Table 10

Agreement of the LLM Coder With the Human Consensus, Against the Human–Human Benchmark

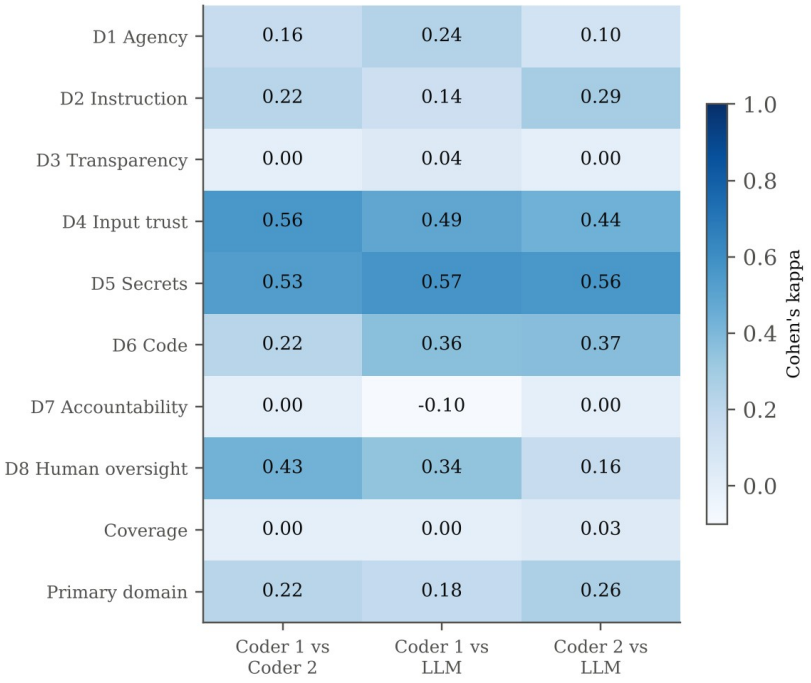
Item	Human–human κ	Cons. n	Cons. positives	Acc.	Prec.	Recall	F1	LLM κ [95% CI]
D1	.16	136	98	.68	.82	.70	.76	.28 [.11, .44]
D2	.22	141	24	.65	.30	.79	.44	.25 [.11, .38]
D3	.00	94	0	1.00	n/a	n/a	n/a	n/a
D4	.56	171	85	.80	.75	.89	.82	.60 [.48, .72]
D5	.53	176	50	.90	.85	.78	.81	.74 [.62, .85]
D6	.22	143	28	.86	.59	.89	.71	.63 [.48, .77]
D7	.00	164	0	.87	.00	n/a	n/a	n/a
D8	.43	189	17	.89	.40	.35	.38	.32 [.09, .53]
Pooled D1–D8	.28	1214	—	.83	.63	.77	.69	**.58 [.53, .63]**
Primary domain	.22	83	—	.46	—	—	—	.30 [.17, .42]

Note. Consensus = cases on which Coder 1 and Coder 2 gave the same label; the LLM is scored against that label. n/a = undefined because the consensus contains no positive cases (both D3 and D7 are coded present by only one human). CIs are percentile bootstrap intervals (2,000 case-level resamples).

Table 10 and Figure 9 answer RQ4. Pooled over the eight domains, the LLM coder agreed with the human consensus on 83.0% of 1,214 domain cells, with κ = .58, 95% CI [.53, .63]. The lower bound is below .60, so $H4_0$ is *not rejected* (bootstrap p = .805). The benchmark puts this result in context. The two human coders agreed with each other at only κ = .28 (95% CI [.23, .33]), and the LLM's pairwise agreement with Coder 2 (κ = .46) was higher than the humans' agreement with each other. At the control level, where both humans judged the same applicable control the same way, the LLM matched them in 82.6% of 407 cells (κ = .65), against human–human control agreement of κ = .43.

Figure 9

Pairwise Cohen's κ Between the Three Coders by Domain



Note. Darker cells show higher agreement. n/a = undefined. Fleiss' κ for the three coders pooled over domains = .34.

The profile across domains is the most useful part of the result. The LLM reached substantial agreement on the domains with concrete textual cues: secrets and data protection (D5, κ = .74), supply-chain integrity (D6, κ = .63), and input trust (D4, κ = .60). It was weak on the judgement-heavy distinctions: agency versus boundary adherence (D1, κ = .28; D2, κ = .25) and human oversight (D8, κ = .32). Its errors were mostly over-coding. It added D2 in 44 consensus-negative cases, often treating an allowlist bypass as both D1 and D2, and it coded D7 in 21 cases where both humans coded it absent. The low agreement is largely a codebook problem rather than a coder problem. Coder 1 coded transparency (D3) in 57.8% of cases and Coder 2 in none, and the two humans disagreed on the primary domain in 63% of cases (κ = .22). Where the codebook wording is concrete, the LLM is as reliable as a trained human. Where it is ambiguous, neither humans nor the LLM converge. This points directly to the Version 2 changes below.

Overall Interpretation

Taken together, the results show that the ACEG Framework is a valid map of disclosed coding-agent weaknesses: it covers almost every case (RQ1), and its control profile is the strongest available predictor of severity. A random forest on that profile reaches a test AUC of .84, against .70 for weakness-type triage (RQ3). Three factors explain the performance. First, severity rises with breadth: the number of failing domains more than doubles the odds per domain. Second, a few controls carry most of the signal: privilege scope (C01), automatic execution of repository configuration (C14), untrusted-input isolation (C08), and egress limits (C11). Third, the ratio-based GCS dilutes that signal, so it neither confirms nor refutes H3 and should not be used alone as a risk score. RQ2 shows that human approval is not tied to any one kind of failure, and RQ4 shows that LLM coding is good enough for concrete domains but needs human review for judgement-heavy ones. Small test-set size and modest human agreement mean these estimates have wide intervals, which the next section discusses.

Practical Significance

The results support four decisions in practice. First, *control priorities:* organizations adopting coding agents should first enforce least-privilege credentials, block automatic execution of repository-level configuration (including Git hooks, MCP definitions, and editor settings), isolate untrusted inputs, and limit network egress. These are the controls whose failure most often goes with high-impact weaknesses. Second, *triage:* security teams can rank incoming agent advisories by the model's predicted probability. At the default threshold this clears about half of the low-impact cases that a CVSS-free rule would escalate, and every probability comes with the specific breached controls. Third, *automation with oversight:* the ACEG Governance Skill packages this triage for industry use; its LLM step can handle first-pass coding of D4, D5, and D6 at about human reliability, while D1, D2, D3, D7, and D8 still

need human review. Fourth, *framework revision:* the evidence supports ACEG Version 2 (Table B2). It merges D3 and D7 into one transparency and accountability domain with explicit triggers, adds a decision rule that separates D1 (approval bypass) from D2 (bypass of a configured limit), requires coders to name the control that would have prevented the weakness, and adds two controls, C21 (authenticated, origin-checked local agent endpoints, prompted by 15 cases of exposed hypertext transfer protocol [HTTP] or WebSocket services) and C22 (transport and host verification for remote sessions, prompted by CA-121).

Implementation and User Benefit

The study's outputs are packaged so that a team can use them on the next advisory without re-running the research pipeline. The industry solution is the ACEG Governance Skill: a self-contained skill package that an LLM agent (for example, Claude with Agent Skills, or any agent framework that can read instructions and run Python) loads when a user asks it to assess an AI coding-agent advisory. A Flask scorecard service exposes the same scoring logic to non-agent systems.

Deployment Approach

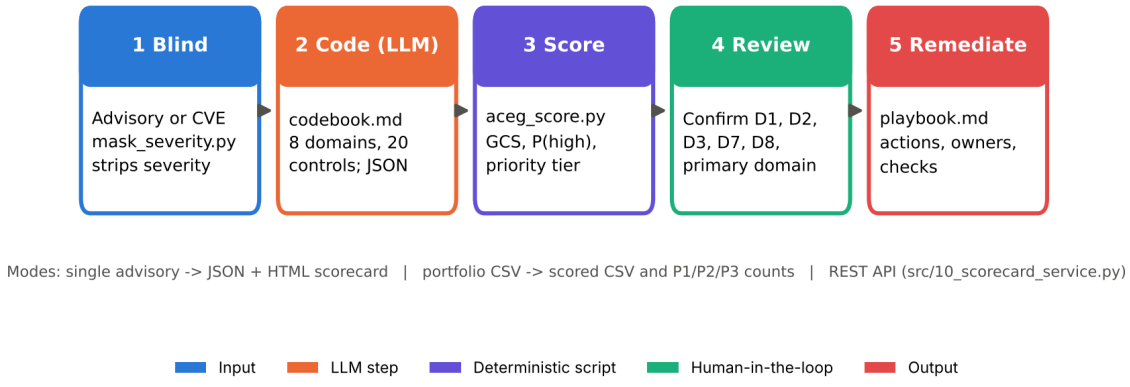
The primary deployment is the ACEG Governance Skill (skill/aceg-governance/ in the repository). It follows the open Agent Skills format: a SKILL.md file with a name and a trigger description, so that an agent loads it only when an advisory, CVE, or incident about a coding agent is in scope, plus reference files, scripts, and model assets (Table 11). Figure 10 shows its five-step workflow. Step 1 collects the advisory and removes severity wording with mask_severity.py, the same blinding used in the study. Step 2 has the LLM apply references/codebook.md, which adds the Version 2 decision rules for the low-agreement domains, and return schema-valid JSON with an evidence quote for each domain. Step 3 runs aceg_score.py, which computes the GCS and the predicted probability of high impact. Step 4 enforces the RQ4 review rule by flagging D1, D2, D3, D7, D8 and the primary domain for

human confirmation. Step 5 builds a prioritized remediation plan from references/controls_playbook.md, giving each breached control concrete actions, an owner, and a verification check.

The skill is designed to run anywhere. aceg_score.py uses the random forest (M7) when scikit-learn can load it and otherwise falls back automatically to portable_model.json, an L2-penalized logistic model stored as plain coefficients that needs only the Python standard library (cross-validated AUC = .84). Its priority thresholds are calibrated to reproduce the forest's tier shares. The script runs in three modes: a single advisory (JSON and HTML scorecard), a portfolio file (CSV in, scored CSV and tier counts out), and the Flask REST service (src/10_scorecard_service.py), which exposes GET /health, POST /score, and GET /scorecard/<case_id> for ticketing and reporting systems. Nothing requires a GPU, and a case is scored in milliseconds on a single CPU core, so the skill and service can run on an analyst's laptop, inside an agent platform, or as a container on Azure App Service, AWS Lambda, or Kubernetes.

Figure 10

Workflow of the ACEG Governance Skill



Note. Colours mark the type of step. The LLM performs only step 2; scoring, tiering, and the remediation plan are deterministic, and step 4 keeps a human in the loop for the domains where LLM agreement was weak (RQ4).

Table 11

Components of the ACEG Governance Skill

File	Purpose
SKILL.md	Trigger description, five-step workflow, output format, interpretation rules (for example, never use GCS alone)
references/codebook.md	Eight domains, 20 controls, OWASP and NIST crosswalk, Version 2 decision rules, extension controls C21 and C22, worked example
references/controls_playbook.md	Remediation actions, owner role, and verification check for each control C01 to C22
references/evidence_base.md	Empirical priors from the 223-case study: prevalence, breach rates, severity drivers, LLM reliability
references/output_schema.json	JSON schema for a coded case (domains, primary domain, controls, coverage, evidence, confidence)
scripts/mask_severity.py	Removes CVSS scores, vectors, and severity words before coding (blinding)
scripts/aceg_score.py	Scores one case or a CSV portfolio; outputs JSON, HTML scorecard, or scored CSV; validates inputs
assets/aceg_risk_model.pkl; assets/portable_model.json	Random forest M7 (refitted on $N = 223$) and dependency-free logistic fallback with calibrated tiers
assets/example_case.json; assets/example_batch_llm_coded.csv	Worked single case (CA-169) and the 223 LLM-coded cases for portfolio testing

The skill was validated as an industry user would run it, with the LLM doing all of the coding and no human labels. On the 56 held-out test cases, scoring the LLM coder's labels with the hold-out random forest gave AUC = .81, 95% CI [.68, .92], balanced accuracy = .71, and recall = .90. That is close to the AUC of .84 obtained with the majority labels and well above weakness-type triage (.70). The portable fallback reached AUC = .81 on the same automated path. The priority tiers also held up out of sample: 18 of 20 Priority 1 cases (90%) were rated High or Critical, compared with 17 of 23 Priority 2 cases (74%) and 4 of 13 Priority 3 cases (31%). The tiers therefore separate urgent from routine work (outputs/final/tables/skill_evaluation.csv and skill_tier_validation.csv).

System Integration

The service fits existing vulnerability-management workflows at three points. In *intake*, a scheduled job subscribes to the GitHub Advisory Database or the CVE feed, filters for the organization's approved agents and extensions, passes each new record to an agent running the ACEG Governance Skill, or calls /score directly with records that are already

coded. In *ticketing*, the returned JSON (priority tier, breached controls, OWASP and NIST crosswalk) maps directly to fields in Jira or ServiceNow, so each breached control becomes a remediation subtask with an owner. In *governance reporting*, scorecards accumulate in the SQLite database or a data warehouse and feed the ISO/IEC 42001 management review and the NIST AI RMF measure function with control-level trends. Because inputs and outputs are plain JSON, no change is needed in the agents themselves.

User Interaction

Users interact through a simple loop. Most users simply ask their agent, for example, “Assess this Cursor advisory with ACEG,” and paste or link the advisory; the skill does the rest. Programmatic users supply a coded record listing the domains present, the primary domain, the breached controls, the CWE identifiers, the component type, and the year. The output is a one-page scorecard (Figure 11) and a remediation plan showing the GCS, the number of domains and the family, the predicted probability of high impact against the portfolio base rate, a priority tier (Priority 1, fix before agent use, at $p \geq .80$; Priority 2, fix this sprint, at .50 to .80; Priority 3, routine patch cycle, below .50), the breached controls with their OWASP and NIST mappings, the playbook actions with owners and verification checks, and the codes flagged for human review. Analysts can query the same results through the API or the SQLite views.

Benefits to Users

Operational efficiency: the scorecard replaces manual reading of every agent advisory with a ranked queue. Because the model clears about half of the low-impact cases that a uniform escalation rule would send to the top of the queue, security engineers' effort goes where severity is concentrated, and LLM pre-coding substantially reduces the time spent coding each case. *Financial impact:* prioritizing the few controls linked to severe weaknesses, such as least privilege and blocking automatic configuration execution, lowers

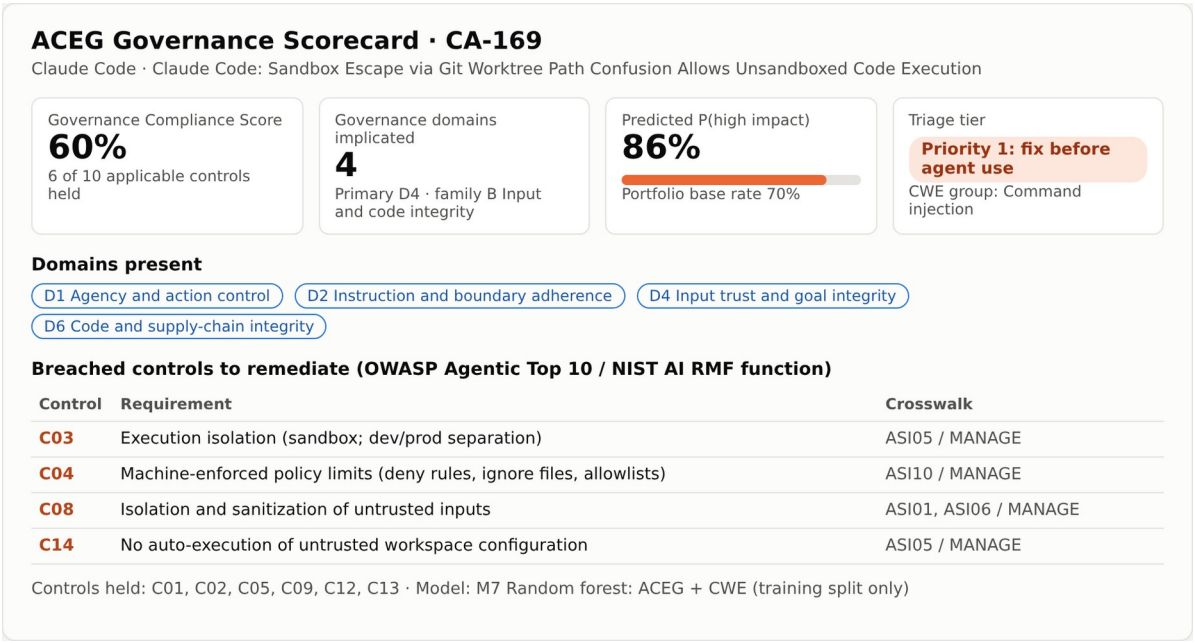
the expected cost of agent-driven incidents like the Replit database deletion and avoids spending on controls with little evidence behind them. *Strategic value:* executives and boards get an auditable, framework-aligned view of agent risk across vendors, which supports procurement decisions, ISO/IEC 42001 certification evidence, and the governance commitments regulators and customers increasingly expect.

Example Use Case

Consider a platform team that has approved Claude Code for its developers. A new advisory appears (CA-169, held out of model training): a sandbox escape through Git worktree path confusion that, when a developer opens a repository containing prompt-injection content, lets the agent overwrite shell start-up files and run code outside the sandbox. The nightly intake agent loads the ACEG Governance Skill, masks the severity wording, codes the advisory, and scores it with the hold-out model. Figure 11 shows the scorecard. Four domains are implicated (agency, boundary adherence, input trust, and supply-chain integrity). Six of ten applicable controls held (GCS = 60%), the predicted probability of high impact is 86%, against a 70% base rate, and the case is placed in Priority 1. The four breached controls, C03 execution isolation, C04 policy limits, C08 untrusted-input isolation, and C14 no auto-execution of workspace configuration, become four tickets, each carrying the playbook action and verification check. For C14, for example, the action is to disable Git hooks and fsmonitor when an agent opens a repository, and the check is that a booby-trapped repository runs nothing. The skill flags D1, D2, and the primary domain for analyst confirmation before the tickets are released. The team pins the patched version, adds a policy that blocks agents from running in repositories that have not been explicitly trusted, and records the action for its ISO/IEC 42001 audit trail. The published CVSS rating, which neither the coders nor the model saw, is High, matching the prediction.

Figure 11

ACEG Governance Scorecard for the Example Use Case (CA-169)



Note. Rendered by GET /scorecard/CA-169 using the model trained on the training split only, so the case is out of sample. ASI = OWASP Agentic Security Initiative risk code; MANAGE = NIST AI RMF function.

Limitations and Further Improvements

The findings rest on public disclosures, a small number of low-impact cases, and coders who agreed only moderately. These constraints are set out below, together with their effects and the work that would address them.

Limitations

Data limitations. Advisories reflect disclosure behaviour as well as product risk. Vendors with active security programmes, such as Anthropic, Cursor, and Microsoft, publish more advisories, so counts by product are not safety rankings. CVSS measures potential technical impact rather than realized or ethical harm, and the score–vector audit showed that different scoring organizations disagree on 43 of 132 version 3.x records. Almost all cases (99%) date from 2025 onward, and one-sentence CVE records (26 cases under 200 characters) give coders little to work with. *Methodological and model limitations.* The two human coders agreed weakly (pooled $\kappa = .28$), and the final labels are majority votes that

include the LLM, so the labels carry measurement error. The data are observational, so the results show associations, not causes. With 223 cases and a 56-case test set, AUC intervals are wide (about ±.10), and the differences among the tree-based models are not statistically distinguishable. The MLP overfit despite regularization, and the random forest is less transparent than logistic regression, although permutation importance mitigates this.

Impact of Limitations

These limitations affect the findings in specific ways. Label noise in D1, D2, D3, D7, and D8 weakens any relationship involving those domains, which biases the RQ2 and RQ3 tests toward the null. The nonsignificant GCS result ($p = .057$) is therefore weak evidence of no effect, not strong evidence of absence. The coverage and prevalence estimates (RQ1) and the tree-model performance are robust across label sources. Disclosure bias limits generalization to products that publish advisories, and the concentration in 2025 and 2026 means the model reflects the current generation of agents, so its calibration should be rechecked as products and scoring practice change. The RQ4 consensus evaluation leaves out the hardest cases, the ones the humans disagreed on, so it may overstate the LLM's agreement on a fresh sample. In the other direction, the low human benchmark means that $\kappa > .60$ may be an unrealistic target until the codebook is revised.

Future Improvements

Four improvements follow directly. First, apply ACEG Version 2 (Table B2), run a calibration round of 20 to 30 cases with a third adjudicating coder, and recode the full data set. Human κ should rise above .60 before the LLM is benchmarked again. Second, add exploitation evidence, such as the Exploit Prediction Scoring System and known-exploited-vulnerability lists, as alternative outcomes that are less tied to one organization's CVSS judgement. Third, model the four-level severity with ordinal and mixed-effects models that treat product as a random effect. Fourth, give the skill's LLM step few-shot examples from

adjudicated cases for the weak domains, and re-validate the skill each quarter against newly disclosed advisories so that its calibration keeps pace with new agent releases.

Future Scope

The approach extends naturally in several directions. The data set can be refreshed quarterly to track whether governance failures shift as vendors adopt sandboxes and permission systems. The framework can be applied to other agent types, such as browser, data-analysis, and customer-service agents, and to incident reports and internal security reviews beyond public advisories. The ACEG Governance Skill can be published to organizational and public skill catalogues, extended with connectors that pull advisories from vendor feeds automatically, and adapted to other agent classes through new codebook modules. The scorecard can be linked to runtime telemetry so that control breaches are detected in agent logs before a vulnerability is disclosed. The researcher plans to continue this line of work at the doctoral level, testing whether organizations that implement the ACEG priority controls experience fewer and less severe agent incidents.

References

Allodi, L., & Massacci, F. (2014). Comparing vulnerability severity and exploits using case-control studies. *ACM Transactions on Information and System Security, 17*(1), Article 1. https://doi.org/10.1145/2630069

Box, G. E. P., & Tidwell, P. W. (1962). Transformation of the independent variables. *Technometrics, 4*(4), 531–550. https://doi.org/10.1080/00401706.1962.10490038

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Brier, G. W. (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review, 78*(1), 1–3. https://doi.org/10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

Cemri, M., Pan, M. Z., Yang, S., Agrawal, L. A., Chopra, B., Tiwari, R., Keutzer, K., Parameswaran, A., Klein, D., Ramchandran, K., Zaharia, M., Gonzalez, J. E., & Stoica, I. (2025). *Why do multi-agent LLM systems fail?* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2503.13657

Chan, A., Salganik, R., Markelius, A., Pang, C., Rajkumar, N., Krasheninnikov, D., Langosco, L., He, Z., Duan, Y., Carroll, M., Lin, M., Mayhew, A., Collins, K., Molamohammadi, M., Burden, J., Zhao, W., Rismani, S., Voudouris, K., Bhatt, U., . . . Maharaj, T. (2023). Harms from increasingly agentic algorithmic systems. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency* (pp. 651–666). Association for Computing Machinery. https://doi.org/10.1145/3593013.3594033

Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement, 20*(1), 37–46. https://doi.org/10.1177/001316446002000104

Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.

Cramér, H. (1946). *Mathematical methods of statistics*. Princeton University Press.

CVE Program. (2026). *CVE List V5* [Data set]. GitHub. Retrieved October 11, 2026, from https://github.com/CVEProject/cvelistV5

Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. (2024). AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. *Advances in Neural Information Processing Systems, 37*, 82895–82920.

Efron, B., & Tibshirani, R. J. (1993). *An introduction to the bootstrap*. Chapman & Hall.

Fleiss, J. L. (1971). Measuring nominal scale agreement among many raters. *Psychological Bulletin, 76*(5), 378–382. https://doi.org/10.1037/h0031619

Forum of Incident Response and Security Teams. (2019). *Common Vulnerability Scoring System v3.1: Specification document*. https://www.first.org/cvss/v3-1/specification-document

Forum of Incident Response and Security Teams. (2023). *Common Vulnerability Scoring System version 4.0: Specification document*. https://www.first.org/cvss/v4-0/specification-document

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics, 29*(5), 1189–1232. https://doi.org/10.1214/aos/1013203451

Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences, 120*(30), Article e2305016120. https://doi.org/10.1073/pnas.2305016120

GitHub. (2026). *GitHub Advisory Database* [Data set]. Retrieved October 11, 2026, from https://github.com/github/advisory-database

Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security* (pp. 79–90). Association for Computing Machinery. https://doi.org/10.1145/3605764.3623985

Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why do tree-based models still outperform deep learning on typical tabular data? *Advances in Neural Information Processing Systems, 35*, 507–520.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly, 28*(1), 75–105. https://doi.org/10.2307/25148625

Hosmer, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression* (3rd ed.). Wiley. https://doi.org/10.1002/9781118548387

Hou, X., Zhao, Y., Wang, S., & Wang, H. (2025). *Model Context Protocol (MCP): Landscape, security threats, and future research directions* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2503.23278

Hsieh, F. Y., Bloch, D. A., & Larsen, M. D. (1998). A simple method of sample size calculation for linear and logistic regression. *Statistics in Medicine, 17*(14), 1623–1634. https://doi.org/10.1002/(SICI)1097-0258(19980730)17:14<1623::AID-SIM871>3.0.CO;2-S

International Organization for Standardization & International Electrotechnical Commission. (2023). *Information technology—Artificial intelligence—Management system* (ISO/IEC Standard No. 42001:2023). https://www.iso.org/standard/81230.html

Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2024). SWE-bench: Can language models resolve real-world GitHub issues? In *Proceedings*

of the Twelfth International Conference on Learning Representations. https://openreview.net/forum?id=VTF8yNQM66

Landis, J. R., & Koch, G. G. (1977). The measurement of observer agreement for categorical data. *Biometrics, 33*(1), 159–174. https://doi.org/10.2307/2529310

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444. https://doi.org/10.1038/nature14539

McGregor, S. (2021). Preventing repeated real world AI failures by cataloging incidents: The AI Incident Database. *Proceedings of the AAAI Conference on Artificial Intelligence, 35*(17), 15458–15463. https://doi.org/10.1609/aaai.v35i17.17817

McHugh, M. L. (2012). Interrater reliability: The kappa statistic. *Biochemia Medica, 22*(3), 276–282. https://doi.org/10.11613/BM.2012.031

National Institute of Standards and Technology. (2023). *Artificial intelligence risk management framework (AI RMF 1.0)* (NIST AI 100-1). U.S. Department of Commerce. https://doi.org/10.6028/NIST.AI.100-1

OWASP Gen AI Security Project. (2025). *OWASP top 10 for agentic applications for 2026*. OWASP Foundation. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/

Page, M. J., McKenzie, J. E., Bossuyt, P. M., Boutron, I., Hoffmann, T. C., Mulrow, C. D., Shamseer, L., Tetzlaff, J. M., Akl, E. A., Brennan, S. E., Chou, R., Glanville, J., Grimshaw, J. M., Hróbjartsson, A., Lalu, M. M., Li, T., Loder, E. W., Mayo-Wilson, E., McDonald, S., . . . Moher, D. (2021). The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ, 372*, Article n71. https://doi.org/10.1136/bmj.n71

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. In *2022*

IEEE Symposium on Security and Privacy (pp. 754–768). IEEE. https://doi.org/10.1109/SP46214.2022.9833571

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Peduzzi, P., Concato, J., Kemper, E., Holford, T. R., & Feinstein, A. R. (1996). A simulation study of the number of events per variable in logistic regression analysis. *Journal of Clinical Epidemiology, 49*(12), 1373–1379. https://doi.org/10.1016/S0895-4356(96)00236-3

Perry, N., Srivastava, M., Kumar, D., & Boneh, D. (2023). Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security* (pp. 2785–2799). Association for Computing Machinery. https://doi.org/10.1145/3576915.3623157

Responsible AI Collaborative. (2025). *Incident 1152: LLM-driven Replit agent reportedly executed unauthorized destructive commands during code freeze, leading to loss of production data*. AI Incident Database. https://incidentdatabase.ai/cite/1152/

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, *323*(6088), 533–536. https://doi.org/10.1038/323533a0

Shavit, Y., Agarwal, S., Brundage, M., Adler, S., O'Keefe, C., Campbell, R., Lee, T., Mishkin, P., Eloundou, T., Hickey, A., Slama, K., Ahmad, L., McMillan, P., Beutel, A., Passos, A., & Robinson, D. G. (2023). *Practices for governing agentic AI systems* [White paper]. OpenAI. https://cdn.openai.com/papers/practices-for-governing-agentic-ai-systems.pdf

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems, 30*, 5998–6008.

Wilson, E. B. (1927). Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association, 22*(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). SWE-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems, 37*, 50528–50652.

Zhan, Q., Liang, Z., Ying, Z., & Kang, D. (2024). InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In *Findings of the Association for Computational Linguistics: ACL 2024* (pp. 10471–10506). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.findings-acl.624

Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems, 36*, 46595–46623.

Appendix A

Data Dictionary

Table A1 defines the source and coded variables used in the analysis (docs/data_dictionary.csv in the repository). The same 30 coded fields (d1 to d8, primary_domain, c01 to c20, covered) are stored for Coder 1, Coder 2, the LLM coder, and the final labels in the four codes_* tables of data/aceg.db.

Table A1

Data Dictionary of Source and Coded Variables

Variable	Type	Definition	Allowed values / units	Missing-value handling	Role
case_id	Identifier	Unique case code	CA-001 to CA-223	None missing	Record key
source	Nominal	Database where the record was found	GHSA; CVE list; both	None missing	Provenance
ghsa_id	Identifier	GitHub advisory identifier	Text	77 structurally missing (CVE-only)	Traceability
cve_id	Identifier	CVE identifier	Text	4 structurally missing (GitHub-only)	Traceability; merge key
url	Text	Link to the public record	URL	None missing	Verification
published	Date	Publication date	YYYY-MM-DD	None missing	Source of year
year	Discrete	Publication year	2023 to 2026	None missing	RQ3 covariate (year_2026)
product	Nominal	Coding agent concerned	20 categories	None missing	Descriptive; RQ3 clusters
component_type	Binary	Agent product or ecosystem component	agent product = 0; ecosystem = 1	None missing	RQ3 covariate
cvss_severity	Ordinal	Published CVSS severity	Low, Medium, High, Critical	None missing	Source of high_impact
cvss_score	Continuous	Published CVSS base score	0.0 to 10.0 points	31 missing: 24 recomputed from v3.x vector; 7 left missing	Sensitivity analysis
cvss_vector	Text	CVSS vector string	Text	2 missing; excluded from RQ2 only	Source of user interaction and version
cvss_version	Binary	CVSS version of the rating	0 = 3.x; 1 = 4.0	None missing	RQ3 sensitivity
high_impact	Binary	Severity High or Critical (base score >= 7.0)	0 / 1	None missing	RQ3 dependent variable
user_interaction_required	Binary	Exploitation needs a human action (CVSS UI)	0 / 1	2 missing; excluded from RQ2 only	RQ2 independent variable
cwe_ids	Text	Weakness types (CWE)	Text	None missing	Descriptive
d1 to d8	Binary	Governance domains present	0 / 1	None missing	RQ1; RQ4
n_domains	Count	Number of domains present	0 to 8	None missing	RQ3 covariate
primary_domain	Nominal	Domain that most directly enabled the weakness	1 to 8	None missing	RQ4 label
domain_family	Nominal	Family of the primary domain	A to D	From final primary domain	RQ2 dependent variable

Variable	Type	Definition	Allowed values / units	Missing-value handling	Role
c01 to c20	Categorical	Status of each control	1 = breached; 0 = not breached; NA = not applicable	NA when the control's domain is absent	GCS input; RQ3 features
n_applicable	Count	Controls linked to the domains present	0 to 20	None missing	GCS denominator
n_breached	Count	Applicable controls breached	0 to 20	None missing	GCS input
gcs	Continuous	Governance Compliance Score	0 to 100 percent	Defined for all 223 cases in final labels	RQ3 independent variable
gcs_z	Continuous	Standardized GCS (computed in analysis)	Standard deviations	None missing	RQ3 model
covered	Binary	Framework covers the case	0 / 1	None missing	RQ1 dependent variable
llm_d1 to llm_d8	Binary	Domains predicted by the pipeline	0 / 1	None missing	RQ4
llm_primary_domain	Nominal	Primary domain predicted by the pipeline	1 to 8	None missing	RQ4
llm_c01 to llm_c20	Categorical	Control status predicted by the pipeline	1; 0; NA	None missing	RQ4 (GCS agreement)
llm_gcs	Continuous	GCS from pipeline codes	0 to 100 percent	None missing	RQ4
llm_model	Text	Model name and version	Text	None missing	Reproducibility

Note. Variables prefixed llm_ hold the LLM coder's labels; the human coders' labels are stored in codes_coder1 and codes_coder2 with the same names. title was missing for 82 CVE-only records and was filled with the first 120 characters of the description.

Appendix B

ACEG Framework: Version 1 Controls and Evidence-Based Version 2 Changes

Table B1 lists the ACEG Version 1 domains, families, and controls with their crosswalk to the OWASP agentic risk codes (ASI) and NIST AI RMF functions, together with the number of cases in which each control was applicable and breached under the final labels. Table B2 records the changes proposed for Version 2 and the evidence behind each change (docs/framework_changelog.md).

Table B1

ACEG Version 1 Domains, Controls, Crosswalk, and Breach Frequency (Final Labels)

Domain (family)	Code	Control	OWASP ASI / NIST AI RMF	Applicable n	Breached n (%)
D1 Agency and action control (A)	C01	Least-privilege credentials and tool permissions	ASI02, ASI03 / MANAGE	149	64 (43%)
	C02	Human approval gate for sensitive or irreversible actions	ASI02 / MANAGE	149	72 (48%)
	C03	Execution isolation (sandbox; dev/prod separation)	ASI05 / MANAGE	149	31 (21%)
D2 Instruction and boundary adherence (A)	C04	Machine-enforced policy limits (deny rules, ignore files, allowlists)	ASI10 / MANAGE	79	65 (82%)
	C05	Workspace trust and scope confirmation before execution	ASI01 / MAP	79	15 (19%)
D3 Transparency and honest reporting (D)	C06	Faithful disclosure of planned actions	ASI09 / GOVERN	6	6 (100%)
	C07	Tamper-evident action logs	ASI10 / GOVERN	6	0 (0%)
D4 Input trust and goal integrity (B)	C08	Isolation and sanitization of untrusted inputs	ASI01, ASI06 / MANAGE	115	111 (96%)
	C09	Allow-listed, integrity-checked tools and MCP servers	ASI04 / MANAGE	115	26 (23%)
D5 Secrets and data protection (C)	C10	Secret scanning, redaction, protected credential storage	ASI03 / MANAGE	67	45 (67%)
	C11	Network egress limits and data-handling policy	ASI02 / GOVERN	67	35 (52%)
D6 Code and supply-chain integrity (B)	C12	Dependency, extension, and release integrity verification	ASI04 / MANAGE	55	13 (24%)
	C13	Security testing gate before release or merge	ASI05 / MEASURE	55	2 (4%)
	C14	No auto-execution of untrusted workspace configuration	ASI05 / MANAGE	55	45 (82%)
D7 Accountability and recoverability (D)	C15	Named human owner and authorization for agent actions	ASI08 / GOVERN	3	2 (67%)
	C16	Reversible actions (checkpoints, backup, rollback)	ASI08 / MANAGE	3	0 (0%)
	C17	Incident response, patching, and disclosure process	ASI08 / MANAGE	3	1 (33%)
D8 Human oversight and reliance (A)	C18	Autonomy level matched to risk (safe defaults)	ASI09 / MAP	22	22 (100%)
	C19	Human review of agent-authored changes and commands	ASI09 / MANAGE	22	5 (23%)
	C20	Capability and limitation disclosure to users	ASI09 / GOVERN	22	0 (0%)

Note. GCS = 100 × (applicable − breached) / applicable. Families: A agency and oversight; B input and code integrity; C data protection; D transparency and accountability.

Table B2

Proposed ACEG Version 2 Changes and Supporting Evidence

Change	Evidence	Source RQ
Merge D3 and D7 into one domain, Transparency and accountability, with explicit triggers (UI hides arguments; documentation misstates a safety property; vendor did not respond or patch)	Human–human κ = .00 for D3 and D7; Coder 1 coded D3 in 57.8% of cases, Coder 2 in 0%; final prevalence 2.7% and 1.3%	RQ4, RQ1
Decision rule separating D1 and D2: bypass of a configured limit (allowlist, deny rule, ignore file, path boundary) is D2 and primary; an action run without approval is D1; code both only when both are stated	κ = .16 (D1) and .22 (D2); LLM over-coded D2 in 44 consensus-negative cases; primary-domain κ = .22	RQ4
Coverage must name the preventing control (evidence field)	Coder 1 coded covered = 1 for every case (no variance; κ undefined)	RQ1, RQ4
Add C21 Authenticated, origin-checked local agent endpoints (HTTP/WebSocket)	15 cases expose agent servers without authentication or origin checks (for example CA-070, CA-117, CA-214)	RQ1
Add C22 Transport and host verification for remote agent sessions	Only uncovered case (CA-121, SSH host-key check)	RQ1
Keep C01, C08, C11, C14 as priority controls in the scorecard	Top permutation-importance controls in the best model (Figure 6)	RQ3
Review C07, C16, C20 (never breached) and C13, C15, C17 (rarely breached) for merging	0 to 2 breaches each under final labels (Table B1)	RQ1, RQ3
Report GCS together with number of domains and breached controls, never alone	GCS OR unstable across label sources; n_domains OR = 2.28	RQ3

Appendix C

Data Cleaning and Additional Exploratory Analysis

Table C1 summarizes the cleaning actions, Table C2 gives descriptive statistics of the numeric variables, and Figure C1 shows the distribution of CVSS base scores and the cases by product.

Table C1

Data Cleaning Log

Issue	Variables affected	Detection	Treatment	Rationale
Missing values	cvss_score (31; 13.9%)	isna() count	24 recomputed from v3.x vectors (all 24 match the published band); 7 left missing	Deterministic recovery; outcome complete
Missing values	title (82; 36.8%)	isna() count	Filled with first 120 characters of description	Text retained for coders
Missing values	cvss_vector, user_interaction (2)	isna() count	Excluded from RQ2 only (n = 221)	Listwise deletion limited to the analysis needing the field
Duplicates	All rows; cve_id; ghsa_id	duplicated()	0 exact duplicates; cross-source records merged on CVE at screening	One case per vulnerability
Outliers	cvss_score (3 below 3.2)	Tukey fences (Q1 = 6.5, Q3 = 8.7)	Kept	Valid Low ratings
Outliers	description length (26)	Tukey upper fence	Kept; log scale in EDA only	Long advisories are informative
Inconsistency	score vs. vector (43 of 132 v3.x)	Score recomputed from vector	Published severity kept; band differs in 19, outcome in 6	Scorer differences documented
Coding rule breach	Coder 1 control cells (171)	Rule check vs. domain codes	Reset to NA; logged (outputs/final/tables/cleaning_log_coding.csv)	Codebook rule 4

Table C2

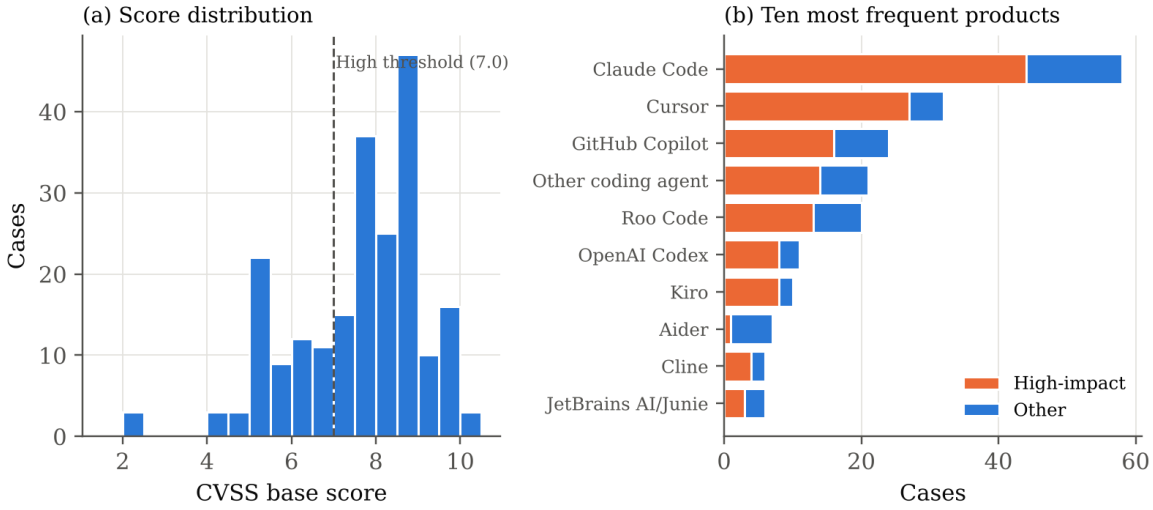
Descriptive Statistics of Numeric Variables

Variable	*n*	*M*	*SD*	Min	Q1	*Mdn*	Q3	Max	Skew
CVSS base score	216	7.57	1.56	2.0	6.5	7.8	8.7	10.0	-0.84
Description length (characters)	223	796.37	722.34	94.0	431.5	588.0	821.5	3000.0	2.10
CWE identifiers per case	223	1.28	0.63	1.0	1.0	1.0	1.0	6.0	3.25
Domains present (final labels)	223	2.22	1.05	1.0	1.0	2.0	3.0	5.0	0.64
Governance Compliance Score (%)	223	52.79	13.74	0.0	50.0	57.1	60.0	66.7	-2.35

Note. M = mean; Mdn = median; Q1 and Q3 = quartiles. CVSS score after recovery from vectors (7 still missing).

Figure C1

Distribution of CVSS Base Scores and Cases by Product



Note. Panel (a): bins of 0.5 points, n = 216; dashed line marks the High threshold. Panel (b): the ten products with the most cases, split by impact class.

Appendix D

Additional Results and Robustness Checks

Table D1 and Figure D1 report the RQ3 sensitivity analyses; Table D2 the full-data cross-validation of all models; Table D3 the pairwise and three-rater agreement for every coded item; Table D4 the RQ2 test under each coder's labels; Figure D2 the RQ2 cross-tabulation; and Figure D3 the LLM coder's precision, recall, and κ against the human consensus.

Table D1

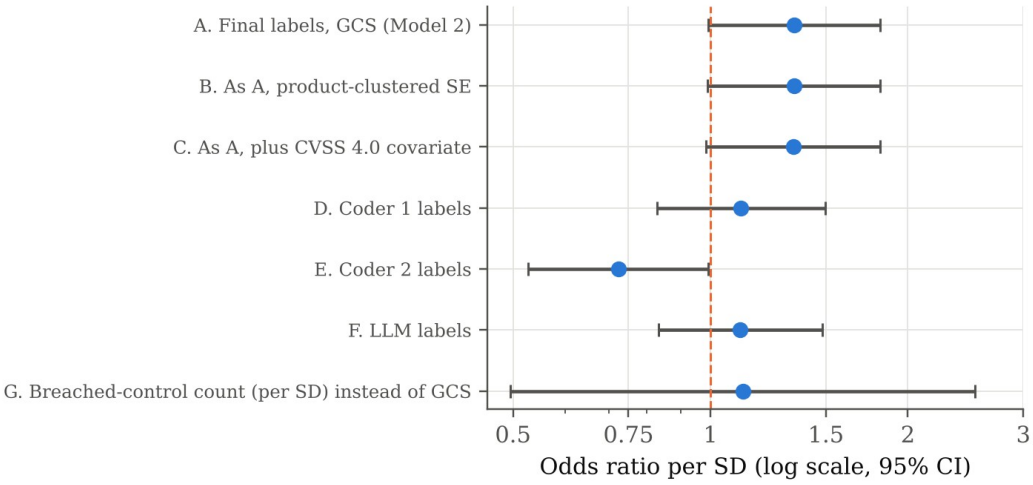
Sensitivity Analyses for the RQ3 Hypothesis Model

Specification	***n***	**OR per *SD***	**95% CI**	***p***	**McFadden R^2**
A. Final labels, GCS (Model 2)	223	1.34	[0.99, 1.81]	.057	.117
B. As A, product-clustered *SE*	223	1.34	[0.99, 1.82]	.058	.117
C. As A, plus CVSS 4.0 covariate	223	1.34	[0.99, 1.82]	.062	.126
D. Coder 1 labels	223	1.11	[0.83, 1.50]	.474	.059
E. Coder 2 labels	223	0.72	[0.53, 0.99]	.045	.046
F. LLM labels	223	1.11	[0.83, 1.48]	.477	.054
G. Breached-control count (per *SD*) instead of GCS	223	1.12	[0.50, 2.53]	.784	.103

Note. All models adjust for ecosystem component, number of domains (from the same label source), and publication in 2026.

Figure D1

Odds Ratio for GCS Across Sensitivity Specifications



Note. Points are odds ratios per *SD*; lines are 95% CIs; the dashed line marks OR = 1 (log scale).

Table D2

Full-Data Repeated Cross-Validation (5 Folds × 10 Repeats, N = 223)

Model	AUC M (*SD*)	Balanced accuracy M (*SD*)
M0 Majority-class baseline	.500 (.000)	.500 (.000)
M1 Logistic: covariates only	.705 (.089)	.581 (.052)
M2 Logistic: GCS + covariates	.712 (.080)	.595 (.064)
M3 Logistic: CWE group (current triage)	.736 (.091)	.581 (.068)
M4 Random forest: ACEG profile	.829 (.057)	.755 (.063)
M5 Gradient boosting: ACEG profile	.810 (.057)	.747 (.067)
M6 Neural network (MLP): ACEG profile	.790 (.065)	.667 (.080)
M7 Random forest: ACEG + CWE	.842 (.059)	.748 (.064)

Note. Hyperparameters as tuned on the training split. Each model is refitted within each of the 50 folds.

Table D3

Prevalence and Pairwise Agreement by Coder

Item	Coder 1 %	Coder 2 %	LLM %	κ C1–C2	κ C1–LLM	κ C2–LLM	Fleiss κ
D1	67.3	59.6	60.5	.16	.24	.10	.17
D2	12.1	46.2	52.5	.22	.14	.28	.16
D3	57.8	0.0	2.7	.00	.04	.00	-.20
D4	38.1	61.4	56.5	.56	.49	.44	.48
D5	27.8	38.1	28.3	.53	.57	.56	.55
D6	43.9	17.0	30.5	.22	.36	.37	.29
D7	26.5	0.0	10.8	.00	-.10	.00	-.10
D8	9.0	21.5	9.0	.43	.34	.16	.29
Coverage	100.0	83.0	99.1	.00	.00	.03	-.04
Primary domain	—	—	—	.22	.18	.26	.20
Pooled domains				.28	.28	.46	.34

Note. κ is 0 or undefined where one coder never used a category (Coder 2 never coded D3 or D7; Coder 1 coded every case as covered).

Table D4

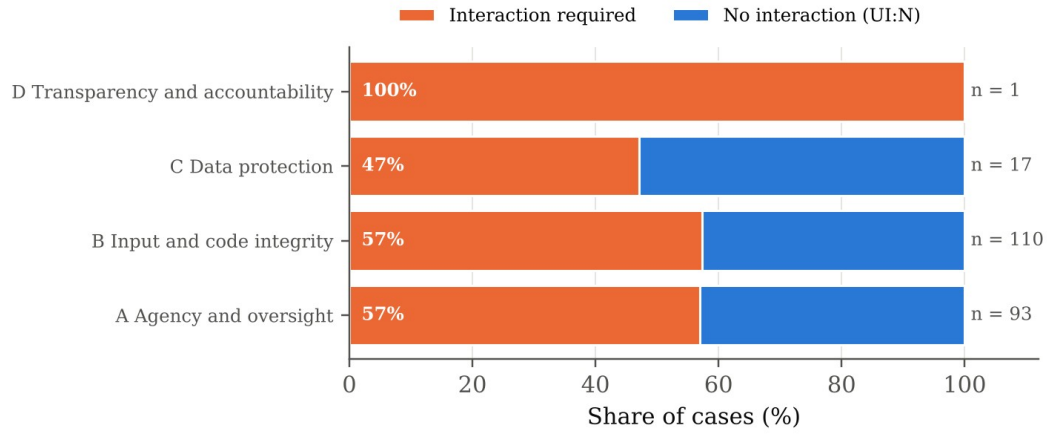
RQ2 Chi-Square Test Under Each Label Source

Labels	χ^2	*df*	*p*	Cramér's V
Final labels	1.42	3	.700	.08
Coder 1	2.89	3	.409	.11
Coder 2	0.90	2	.636	.06
LLM coder	5.75	3	.125	.16

Note. Coder 2 used only three families as primary domains (*df* = 2).

Figure D2

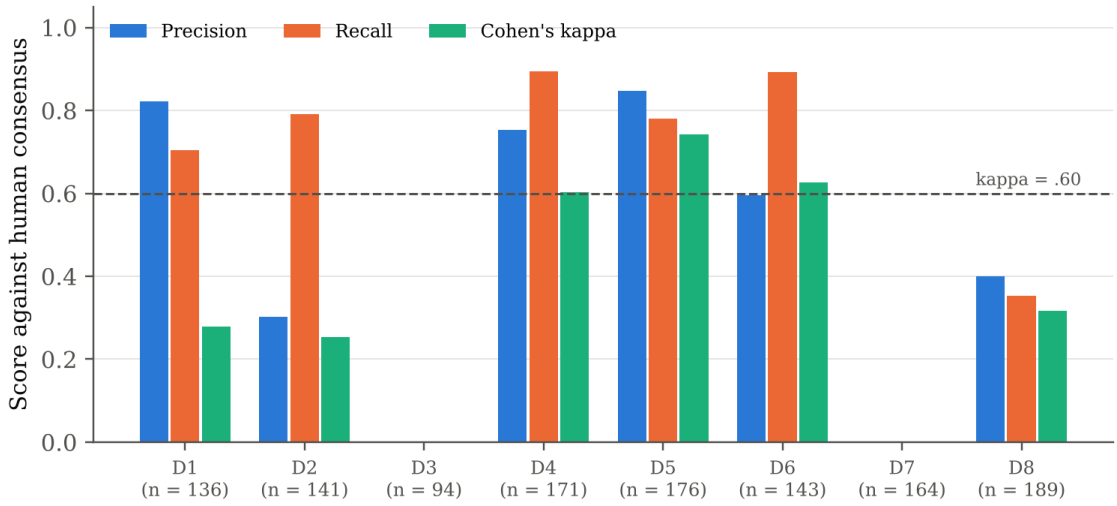
Domain Family of the Primary Domain by Need for User Interaction (RQ2)



Note. n = 221 cases with a CVSS vector. Labels on the bars give the share requiring interaction.

Figure D3

LLM Coder Against the Human Consensus by Domain (RQ4)



Note. n = cases on which both human coders agreed. D3 and D7 have no consensus-positive cases, so precision, recall, and κ are undefined.

Appendix E

Reproduction Steps, Database Queries, and API Example

All results are reproduced from the public repository with the steps below (Python 3.13; about 5 minutes on a 2-core CPU; seed 42). The code used to generate every table and figure is in src/ and tools/; for example, src/09_final_analysis.py generates Figures 1, 2, and 4 to 9, and tools/architecture_diagram.py generates Figure 3.

1. pip install -r requirements.txt; python sample_size/sample_size.py (Table 4).
2. Clone the two sources at the commits in docs/data_freeze.md; python src/01_collect.py (239 records); python src/02_screen.py (223 cases).
3. python src/07_parse_llm_codes.py (expands data/llm/llm_codes_compact.txt to data/processed/llm_codes.csv).
4. python src/08_build_gold_and_db.py (reads coding/completed/*.xlsx, cleans, builds majority labels, writes data/processed/cases_coded.csv and data/aceg.db).
5. python src/09_final_analysis.py (RQ1 to RQ4, models M0 to M7, tables in outputs/final/tables/, figures in outputs/final/figures/, summary.json, models/*.pkl).
6. python src/10_scorecard_service.py serve (REST API on port 8080) or python src/10_scorecard_service.py demo CA-169 (HTML scorecard).
7. python src/11_build_skill_and_evaluate.py (portable model, skill assets, end-to-end skill evaluation in outputs/final/tables/skill_evaluation.csv).
8. Skill use: python skill/aceg-governance/scripts/aceg_score.py skill/aceg-governance/assets/example_case.json --html card.html (single case) or ... example_batch_llm_coded.csv --batch scored.csv (portfolio).
9. python tools/architecture_diagram.py; python tools/skill_diagram.py; python tools/build_report.py (this report).

Example SQL query (data access). The analysis view joins source records and final labels:

```
SELECT c.case_id, c.product, c.high_impact,
       g.gcs, g.n_domains, g.primary_domain
FROM cases AS c JOIN codes_gold AS g USING (case_id)
WHERE c.component_type = 'agent product' AND g.d1 = 1
ORDER BY g.gcs;
```

Example API call (deployment). A coded advisory is posted to the scorecard service, which returns the scorecard as JSON:

```
curl -X POST http://127.0.0.1:8080/score \
  -H "Content-Type: application/json" \
  -d '{"case_id": "NEW-001", "component_type": "agent product",
       "year": 2026, "cwe_ids": "CWE-78", "domains": [1, 2, 4],
       "primary_domain": 2, "breached_controls": [2, 4, 8]}'
Response: {"gcs": 57.1, "n_domains": 3, "p_high_impact": 0.919,
  "risk_tier": "Priority 1: fix before agent use",
  "breached_controls": [{"code": "C02", ...}, {"code": "C04", ...},
                        {"code": "C08", ...}], ...}
```

Appendix F

ACEG Governance Skill: Instructions, Package, and Example Run

The ACEG Governance Skill is the industry deliverable of the project (skill/aceg-governance/ in the repository; Table 11 lists its files). It packages the codebook, the blinding rule, the trained models, the human-review rule from RQ4, and the remediation playbook so that any LLM agent can apply the framework to a new advisory. Its SKILL.md is summarized below. The LLM coder used in RQ4 followed the same coding instructions (Steps 1 and 2).

– Name and trigger: aceg-governance. Use it when assessing a security advisory, CVE, or incident in an AI coding agent or its ecosystem (extensions, MCP servers, hooks, CI workflows) to produce a governance scorecard, high-impact risk estimate, triage priority, and remediation plan. Do not use it for software CVEs that involve no agent.

– Step 1, collect and blind: gather the title, product, component type, year, CWE identifiers, and description; run scripts/mask_severity.py and code only from the masked text.

– Step 2, code: read references/codebook.md; mark each domain D1 to D8 present if its requirement was breached or could be breached; choose one primary domain using the Version 2 decision rules; list breached controls of present domains only; set covered and name the preventing control; quote evidence; return JSON that matches references/output_schema.json.

– Step 3, score: run scripts/aceg_score.py case.json (or a CSV with --batch). It reports the GCS, the number of domains, the family, P(high impact) against the 70% base rate, the priority tier (P1 at .80 or above, P2 from .50, P3 below .50), and the model used (random forest or portable fallback).

– Step 4, human review: accept D4, D5, and D6 after spot checks; flag D1, D2, D3, D7, D8, and the primary domain for confirmation; never present the scorecard as final until a person has confirmed the flagged codes.

– Step 5, remediate and report: for each breached control, take the actions, owner, and verification check from references/controls_playbook.md; list the priority controls C01, C14, C08, C11, and C02 first; log any coverage gap for the next framework version.

– Interpretation rules: never use GCS alone as a risk score (always report it with the number of domains and the breached controls); P(high impact) estimates CVSS-style technical severity, not exploit likelihood or business impact; advisory counts by vendor are not safety rankings.

Example run (CA-169, single-case mode). The command and an abridged output follow:

```
$ python scripts/aceg_score.py assets/example_case.json --html card.html
{"case_id": "CA-169", "gcs": 60.0, "controls_held": 6, "n_applicable": 10,
 "n_domains": 4, "primary_domain": "D4 Input trust and goal integrity",
 "p_high_impact": 0.873, "priority": "P1 fix before agent use",
 "remediation_plan": [{"control": "C14", "action": "Disable hooks, tasks,
    fsmonitor and .env routing on repository open", ...},
   {"control": "C08", ...}, {"control": "C03", ...}, {"control": "C04", ...}],
 "human_review_required": ["D1", "D2", "primary_domain"],
 "model_used": "M7 Random forest: ACEG + CWE"}
$ python scripts/aceg_score.py assets/example_batch_llm_coded.csv --batch scored.csv
{"cases": 223, "priority_counts": {"P1 fix before agent use": 63,
  "P2 fix this sprint": 96, "P3 routine patch cycle": 64}, "model_used": "forest"}
```

The single-case output uses the random forest refitted on all 223 cases, which is the production model, so CA-169's probability (.873) differs slightly from the out-of-sample value in Figure 11 (.86, from the hold-out model). The portfolio counts come from fully automated LLM coding of all 223 cases.